

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»



Направление подготовки
09.03.01 Информатика и вычислительная техника

направленность (профиль)
«Технологии разработки программного обеспечения»

Выпускная квалификационная работа

Разработка и проектирование модели по генерации сказок на основе
идей Джозефа Кэмпбелла.

Обучающегося 4 курса
очной формы обучения
Маковеева Никиты Владимировича

Руководитель выпускной квалификационной
работы:
кандидат физико-математических наук,
доцент, доцент кафедры ИТиЭО
Власов Дмитрий Викторович

Санкт-Петербург
2026

Оглавление

Глава 1. Теоретические основы и анализ существующих подходов к генерации сказочных нарративов	7
1.1 Архитектура и принципы работы современных языковых моделей: от LLM к SLM	7
1.2 Методы тонкой настройки и эффективной адаптации моделей	9
1.3 Нарративная структура «Пути героя» Дж. Кэмпбелла и её адаптация к сказочному жанру	11
1.4 Обзор существующих решений в области автоматической генерации сюжетов.	15
1.5 Проблематика разметки данных и контроль нарративной последовательности при обучении SLM.	17
Глава 2. Проектирование и реализация системы генерации сказок на базе малой языковой модели	19
2.1 Архитектура программной системы и выбор технологического стека.	19
2.2 Подготовка корпуса данных и стратегия разметки	21
Этап 1: Подготовка данных для базового обучения.....	21
Этап 2: Подготовка данных для дообучения.....	21
2.3 Реализация архитектуры SLM и механизма LoRA.....	23
Интеграция Low-Rank Adaptation (LoRA)	23
2.4 Протокол обучения и оптимизация гиперпараметров	25
Стадия 1: Базовое дообучение	25
Стадия 2: LoRA Fine-Tuning	27
2.5 Механизмы генерации и контроля качества вывода	28
2.6 Оценка эффективности и промежуточные результаты.....	29
2.7. Методические улучшения на этапе дообучения SLM с LoRA.....	30
2.7.1. Переход к аннотированному корпусу с явными структурными маркерами	30
2.7.2. Введение функции потерь со штрафом за повторения	31
2.7.3. Алгоритм поэтапной генерации	31
2.7.4. Настройка гиперпараметров LoRA-адаптеров.....	32
2.7.5. Качественные результаты дообучения	32
2.8. Сравнительный анализ подходов: дообучение SLM против адаптации LLM.....	34

2.8.1. Настройка среды и параметры QLoRA	34
2.8.2. Подготовка инструктивных данных.....	35
2.8.3. Анализ результатов сравнения	36
ГЛАВА 3. Экспериментальные исследования и сравнительный анализ результатов.....	38
3.1. Методология и условия проведения экспериментов	38
3.2. Результаты дообучения малой языковой модели	40
3.3. Результаты параметрической адаптации Qwen2.5-7B-Instruct	41
3.4. Сравнительный анализ SLM и LLM подходов.....	42
3.5. Качественная оценка соответствия нарративной структуре.....	43
3.6. Практическое развёртывание и ограничения системы.....	45
Заключение	46
Список используемых источников	49
Приложения	50
Приложение А. Код базового обучения SLM-модели.	50
Приложение Б. QLoRA для qwen модели.....	67
Приложение В. Модифицированный LoRA для SLM.....	75

ВВЕДЕНИЕ

Актуальность темы исследования обусловлена стремительным развитием технологий генерации естественного языка и растущим спросом на специализированные решения в области искусственного интеллекта.

Большие языковые модели (LLM) демонстрируют высокие показатели качества текстов. Текущий уровень LLM позволяет ей анализировать и генерировать новые тексты с высокой точностью.

Одна из задач языковых моделей - это генерация новых сюжетов. Уже сегодня мы можем видеть как активно применяются языковые модели в этой области. Например в последнее время социальные сети заполнили короткие AI видео, которые собирают миллионы просмотров. Изготовление таких видео, как правило, полностью перенесено на плечи ИИ-агентов, в основе которых лежат языковые модели. Сюжеты для роликов так же генерируются с помощью таких моделей, что позволяет зарабатывать на рынке коротких видео не качеством контента, а его количеством. Учитывая современные тенденции была выбрана тема данной работы.

Степень научной разработанности проблемы характеризуется наличием фундаментальных работ в области архитектуры трансформеров, методов тонкой настройки языковых моделей и исследований автоматической генерации текстов.

Теория «мономифа» Джозефа Кэмпбелла, описывающая универсальные этапы пути героя, доказала свою эффективность при анализе и конструировании сказочных и мифологических текстов. Интеграция данной теории в процесс обучения SLM открывает возможность создания моделей,

которые будут следовать четкой структуре, что позволит увеличить качество текстов.

Значительный вклад в теорию нарративных структур внесли труды В. Проппа, К. Грейвза, Дж. Кэмпбелла и современных исследователей автоматизированного извлечения нарратива из текста .

Однако на текущий момент практически отсутствуют исследования, системно объединяющие формализованную модель «пути героя» с процессом дообучения языковой модели для генерации жанрово-специфичных текстов.

Большинство существующих решений полагаются на промпт-инжиниринг или используют универсальные LLM без явного внедрения нарративных ограничений на уровне архитектуры или обучающей выборки.

Целью данной работы является разработка языковой модели по генерации сказок. Модель должна будет генерировать сказки согласно структуре «мономифа» Джозефа Кэмпбелла.

Объектом исследования выступает процесс автоматической генерации художественных текстов сказочного жанра с применением методов машинного обучения.

Предметом исследования является модель малого размера (SLM), спроектированная и дообученная для генерации сказок, структурно соответствующих этапам «мономифа» Дж. Кэмпбелла. А так же большие языковые модели, дообученные на подготовленном наборе сказок.

Цель работы заключается в разработке и проектировании модели, способной генерировать связные сказочные тексты, последовательно реализующие ключевые нарративные фазы теории Дж. Кэмпбелла.

Для достижения поставленной цели решаются следующие задачи:

1. Провести анализ теории «мономифа» Дж. Кэмпбелла и адаптировать её этапы к структурным особенностям сказочного жанра.[1]
2. Сформировать и предобработать корпус текстов, разметив сказки по ключевым фазам пути героя и подготовив данные для обучения.
3. Реализовать программный прототип модели и провести экспериментальную оценку качества генерации с использованием автоматических метрик и экспертного анализа.
4. Используя Lora для LLM на размеченных данных и с помощью промт-инжиниринга сравнить результаты с SLM. [2]
5. На основе сравнения разработать лучшую модель по генерации сказок.
6. Разработать пользовательский интерфейс для взаимодействия с моделью и демонстрации результатов генерации.

В работе применяются следующие методы исследования:

- системный и сравнительный анализ научной литературы и архитектур моделей;
- методы машинного обучения (fine-tuning, инструктивное дообучение, контроль вывода через структурированные промты и декодинг);
- количественные и качественные метрики оценки текстов (ROUGE/BERTScore, оценка связности и соответствия структуре);[3]
- программное моделирование и экспериментальная верификация.

Теоретическая значимость работы заключается в расширении представлений о возможностях SLM в области контролируемой генерации текстов и создании методического моста между компьютерной лингвистикой и нарратологией.

Практическая значимость определяется возможностью использования разработанной модели и сопутствующей методики в образовательных платформах, интерактивных сервисах по рассказу историй, системах генерации контента для детской и подростковой аудитории, а также в проектах.

Структура работы обусловлена логикой исследования и включает введение, три главы, заключение, список использованных источников и приложения.

1. В первой главе рассматриваются теоретические основы генерации текстов, архитектура SLM и нарративная теория Дж. Кэмпбелла.
2. Во второй главе описывается проектирование модели, подготовка данных, выбор методов обучения и реализация системы.
3. В третьей главе представлены результаты экспериментов, оценка качества генерации и анализ практического применения.
4. Заключение содержит основные выводы и перспективы дальнейшего развития исследования.

Глава 1. Теоретические основы и анализ существующих подходов к генерации сказочных нарративов

1.1 Архитектура и принципы работы современных языковых моделей: от LLM к SLM

Современные языковые модели базируются на архитектуре Transformer, предложенной в работе Vaswani [4], которая заменила рекуррентные и сверточные механизмы на многослойные блоки самовнимания (self-attention).

Благодаря масштабированию параметров (scaling laws) [4], крупные языковые модели (Large Language Models, LLM) продемонстрировали способность к обучению без примеров, генерации связных текстов и решению сложных логических задач. Однако рост объема параметров (от десятков до сотен миллиардов) сопровождается экспоненциальным увеличением требований к вычислительным ресурсам, памяти и энергопотреблению, что ограничивает их применение в проектах с ограниченным бюджетом.

В ответ на эти ограничения сформировалось направление малых языковых моделей (Small Language Models, SLM) с объемом параметров от 1 до 7 млрд. Исследования показывают, что при узконаправленной дообученности и качественной подготовке данных SLM способны демонстрировать результаты, сопоставимые с LLM на специфичных задачах [5]. Ключевые преимущества SLM включают:

- возможность локального развёртывания на потребительском оборудовании (CPU/GPU среднего класса);
- снижение стоимости инференса и хранения весов;

- более предсказуемое поведение при контролируемой генерации благодаря меньшей емкости модели и меньшей склонности к «галлюцинациям» на доменно-специфичных выборках.

Для задачи генерации сказочных текстов выбор SLM обусловлен необходимостью тонкого контроля нарративной последовательности, который на практике часто деградирует в LLM из-за избыточной параметрической свободы и стохастического декодинга. SLM, напротив, при корректной подготовке обучающего корпуса демонстрируют более стабильное следование заданной структуре, что делает их предпочтительной базой для данной работы.

1.2 Методы тонкой настройки и эффективной адаптации моделей

Перенос предобученных языковых моделей в специфичные предметные области осуществляется преимущественно через три стратегии:

1. Полное дообучение (Full Fine-tuning, SFT) – обновление всех весов модели на размеченном корпусе. Обеспечивает максимальную адаптацию, но требует значительных вычислительных ресурсов и сопровождается риском катастрофического забывания (catastrophic forgetting) базовых языковых навыков.
2. Параметрически эффективное дообучение (PEFT) – обновление только части параметров. Наиболее распространённый подход – Low-Rank Adaptation (LoRA) [2], который замораживает исходные веса и обучает низкоранговые матрицы приращений для линейных слоёв. LoRA снижает потребление VRAM на 70–90% при сохранении качества генерации, однако для моделей с малым числом параметров (<3B) его эффективность может снижаться из-за избыточной регуляризации и ограничения адаптационной емкости.
3. Промпт-инжиниринг и инструктивное обучение – использование структурных промптов, примеров и системных инструкций без изменения весов. Метод прост в реализации, но не обеспечивает устойчивого следования сложным нарративным схемам на длинных текстах, так как модель опирается на вероятностное распределение, а не на выученные структурные зависимости.

В контексте данной работы сравнение подходов SFT (для SLM) и LoRA+инструктивные промпты (для LLM) обусловлено необходимостью

эмпирически выявить баланс между структурной строгостью, лингвистическим качеством и вычислительной эффективностью. Выбор конкретного метода будет зависеть от результатов эксперимента, однако теоретический анализ показывает, что для SLM целесообразно использовать легковесное SFT с ранним остановом (early stopping) и регуляризацией, предотвращающей переобучение на шаблонных конструкциях.

1.3 Нарративная структура «Пути героя» Дж. Кэмпбелла и её адаптация к сказочному жанру

Теория мономифа, сформулированная Дж. Кэмпбеллом в работе «Тысячеликий герой», описывает универсальную схему трансформации персонажа через 17 дискретных этапов,

сгруппированных в три макрофазы: Отправление, Инициация и Возвращение [1].

Данная схема сохраняет высокую практическую ценность благодаря своей формализуемости и инвариантности к культурному контексту, что делает её пригодной для алгоритмического контроля генерации текстов [6].

Для интеграции в процесс обучения языковой модели этапы мономифа представлены в виде структурированных тегов, которые используются как маркеры последовательности при подготовке обучающих данных и управлении декодингом. В таблице 1 приведена полная схема с адаптацией к сказочному жанру:

Таблица 1.1 – Этапы пути героя.

Этап (тег)	Макрофаза	Нарративная функция в сказке
<u>stage call to adventure</u>	Отправление	Появление проблемы, запрета или просьбы, инициирующей сюжет
<u>stage refusal of call</u>	Отправление	Сомнение героя, страх, внешнее препятствие

<u>stage supernatural aid</u>	Отправление	Появление дарителя, волшебного помощника или магического артефакта
<u>stage crossing threshold</u>	Отправление	Переход в иной мир (лес, царство) — смена правил повествования
<u>stage belly of whale</u>	Отправление	Точка невозврата, символическое испытание или трансформация
<u>stage road of trials</u>	Инициация	Серия испытаний, загадок, встреч с антагонистами
<u>stage meeting goddess</u>	Инициация	Встреча с мудрым/благожелательным персонажем, получение дара
<u>stage woman as temptress</u>	Инициация	Искушение, отвлечение от цели, моральный выбор
<u>stage atonement father</u>	Инициация	Конфронтация с авторитетом/антагонистом, преодоление страха
<u>stage apotheosis</u>	Инициация	Внутреннее преображение героя, обретение мудрости или силы
<u>stage ultimate boon</u>	Инициация	Получение цели поиска (артефакт, знание, освобождение)
<u>stage refusal return</u>	Возвращение	Нежелание героя возвращаться, новые препятствия
<u>stage magic flight</u>	Возвращение	Бегство или погоня, использование волшебных средств для спасения
<u>stage rescue from without</u>	Возвращение	Помощь извне для завершения возвращения

<u>stage_crossing_return_threshold</u>	Возвращение	Возвращение в обычный мир, интеграция полученного опыта
<u>stage_master_two_worlds</u>	Возвращение	Герой обретает баланс между мирами, становится лидером или наставником
<u>stage_freedom_to_live</u>	Возвращение	Финал: мораль, награда, восстановление порядка, обещание счастья

Каждый тег представляет собой маркер, который может быть использован:

- **На этапе подготовки данных** — как префикс к абзацам текста при формировании обучающих пар «структура → текст»;
- **На этапе обучения** — как часть целевой последовательности, позволяющая модели выучить вероятностные связи между структурным маркером и соответствующим нарративным контентом;
- **На этапе инференса** — как управляющий сигнал в промпте или как условие для условного декодинга, ограничивающее пространство генерации рамками текущей фазы.

Важно отметить, что не все сказки реализуют полную 17-этапную схему: фольклорные тексты часто пропускают второстепенные этапы или объединяют несколько функций в одном эпизоде. Поэтому в процессе разметки допускается *гибкая последовательность*: модель обучается на подмножествах тегов, а контроль последовательности на примере реализуется через *допустимые переходы* (valid transitions), заданные в промте.

Такой подход позволяет:

- Сохранить теоретическую строгость схемы Кэмпбелла

- Сохранить логическую последовательность, но упрощает избыточные этапы, делая схему применимой для автоматической разметки и контроля генерации.

1.4 Обзор существующих решений в области автоматической генерации сюжетов.

Автоматическая генерация сюжетов на естественном языке (NLG) [7] развивается по трём основным направлениям:

- Правило-базированные системы (например, Tale-Spin, Storytron) используют формальные грамматики и конечные автоматы. Обеспечивают строгий контроль структуры, но генерируют шаблонные, лингвистически бедные тексты.
- Гибридные подходы комбинируют нейросетевую генерацию с графами сюжетов или онтологиями персонажей. Пример: система Dramatron [8] Позволяют управлять сюжетной дугой, но требуют ручной конструирования графов и плохо масштабируются.
- Нейросетевые модели (LLM/SLM) генерируют текст полностью на основе вероятностного распределения.

Демонстрируют высокую лингвистическую естественность, но страдают от потери структурного контроля, повторяемости и семантических дрейфов на длинных дистанциях. [9]

В контексте сказочной генерации наиболее перспективными считаются подходы, внедряющие структурные ограничения непосредственно в процесс обучения или декодинга.

Работы [8], [9] показывают, что разметка корпусов по нарративным фазам и обучение моделей с явными маркерами этапов значительно повышают соответствие генерации заданной схеме.

Однако существующие наборы данных (например, Project Gutenberg, StoryBench) не содержат разметки по этапам Кэмпбелла, а ручная аннотация

тысяч текстов трудоёмка. Это создаёт методический пробел, который данная работа призвана частично восполнить. [6]

1.5 Проблематика разметки данных и контроль нарративной последовательности при обучении SLM.

Ключевым барьером при проектировании модели, генерирующей сказки по структуре Кэмпбелла, является отсутствие готовых размеченных корпусов и высокая стоимость ручной аннотации. Сказочные тексты редко содержат явные маркеры этапов пути героя; границы фаз размыты, а некоторые сказки реализуют лишь частичную схему (например, опускают «Возвращение» или «Отказ от зова»).

Прямое обучение SLM на неразмеченных данных приводит к усреднению нарративной дуги и потере структурной последовательности.

В текущей практике применяются следующие стратегии разметки:

- Ручная экспертная аннотация — высокая точность, но низкая масштабируемость. Подходит для пилотных выборок (20–50 текстов).
- LLM-ассистированная разметка — использование сильной модели для сегментации текста по этапам с последующей валидацией. Требует разработки инструкций, проверки на согласованность (inter-annotator agreement) и фильтрации ошибок галлюцинации.
- Слабое обучение — применение эвристик (ключевые фразы, смена локации, появление помощника/антагониста) для автоматической предварительной разметки.

Для контроля нарративной последовательности на этапе генерации применяются:

- Структурированные промпты с явными маркерами этапов (<departure>, <initiation>, <return>);
- Условный декодинг с масками или штрафами для предотвращения пропуска фаз;
- Поэтапная генерация (chain-of-stages), когда модель генерирует текст фаза за фазой с фиксацией промежуточных состояний.

В рамках данной работы будет реализован гибридный подход: формирование базового корпуса через LLM-ассистированную разметку с последующей экспертной верификацией, обучение SLM на размеченных парах «структура → текст», и сравнение с LLM+LoRA, использующей только промпт-инжиниринг.

Это позволит эмпирически оценить, насколько явное внедрение нарративной схемы в процесс обучения компенсирует отсутствие сложных архитектурных модификаций и обеспечивает ли SLM преимущество в структурной строгости при сопоставимом качестве языка.

Глава 2. Проектирование и реализация системы генерации сказок на базе малой языковой модели

2.1 Архитектура программной системы и выбор технологического стека

Разработка системы генерации сказок выполнена в виде модульного Python-приложения, использующего фреймворк PyTorch для реализации нейросетевой архитектуры и библиотеку *transformers* для токенизации. Выбор обусловлен необходимостью низкоуровневого контроля над процессом обучения, поддержкой смешанной точности (*mixed-precision*) и возможностью кастомной реализации механизмов параметрической эффективной настройки (PEFT).

Архитектура системы состоит из следующих ключевых модулей:

- **Модуль данных (Data Module):** отвечает за загрузку, очистку и токенизацию корпуса сказок, формирование батчей для базового обучения и инструктивного дообучения.
- **Модуль модели (Model Module):** реализует архитектуру GPT-2 Small «с нуля», включая слои самовнимания, MLP-блоки и механизмы нормализации.
- **Модуль адаптации (Adaptation Module):** реализует механизм LoRA через регистрацию параметризаций весовых матриц, позволяя замораживать основную часть модели и обучать только низкоранговые приращения.
- **Модуль инференса (Inference Module):** содержит алгоритмы декодинга с применением температурного сэмплирования, Top-K и Top-P (*nucleus*) фильтрации для контроля разнообразия генерируемого текста.

Система развернута в среде Kaggle Notebooks с использованием GPU NVIDIA Tesla T4, что определяет ограничения по видеопамяти (16 ГБ) и диктует необходимость использования градиентной аккумуляции и автоматического масштабирования градиентов (GradScaler).

2.2 Подготовка корпуса данных и стратегия разметки

В качестве источника данных использован датасет *folk-tales-deduplicated.csv*, содержащий более 2900 уникальных сказок различных культурных традиций (греческие, скандинавские, кельтские, русские и др.). Процесс подготовки данных разделен на два этапа, соответствующих стадиям обучения модели.

Этап 1: Подготовка данных для базового обучения

На данном этапе цель модели — усвоить лингвистические паттерны, лексику и стилистику сказочного жанра. Данные обрабатываются классом *FolktaleDataLoader*:

1. Тексты сказок конкатенируются в единую последовательность токенов.
2. Между сказками добавляется специальный токен конца текста (`<|endoftext|>`), чтобы модель научилась распознавать границы повествования.
3. Последовательность разбивается на фиксированные блоки длиной $T=512$ токенов.
4. Формируются пары (x, y) , где y является сдвигом x на один токен вправо (next-token prediction task).

Такой подход позволяет эффективно использовать вычислительные ресурсы, избегая паддинга (padding) коротких сказок, и обеспечивает высокий темп обучения за счет больших эффективных размеров батча.

Этап 2: Подготовка данных для дообучения

Для адаптации модели к генерации по запросу и внедрению структурного контроля используется класс *FolktalePromptDataset*. На этом

этапе реализуется гибридная стратегия разметки, направленная на обучение модели реагировать на контекстные подсказки:

- **Генерация промптов:** Для каждой сказки формируются два типа обучающих примеров:
 - *Генерация с нуля:* Промпт вида "Write a \{nation\} folktale called \{title\}:" сопровождается полным текстом сказки.
 - *Продолжение текста:* Сказка разбивается на предложения, и модель обучается продолжать текст, начиная с случайного фрагмента (контекст + продолжение).
 - *Маскировка промптов:* В целевых последовательностях (labels) токены, относящиеся к инструкции (промпту), маркируются значением -100. Это исключает их из расчета функции потерь (Cross-Entropy Loss), фокусируя обучение исключительно на качестве генерации целевого текста.
- **Обработка длины:** Последовательности обрезаются или дополняются паддингом до фиксированной длины $T=512$, при этом приоритет отдается сохранению целевого ответа, а не инструкции.

В текущей реализации пайплайна разметка по этапам мономифа (теги `<stage_call_to_adventure>` и др.) осуществляется на уровне формирования промптов. Структурные теги могут быть явно включены в инструкцию (например, Write a tale following stages: `<stage_call>...<stage_return>`"), что заставляет модель учиться ассоциировать эти маркеры с соответствующими нарративными блоками в процессе LoRA-дообучения.

2.3 Реализация архитектуры SLM и механизма LoRA

Базовая модель реализована как авторегрессионный трансформер декодерного типа (GPT-2 Small) со следующими гиперпараметрами:

- Количество слоев (n_{layer}): 12
- Размерность скрытого состояния (n_{embed}): 768
- Количество голов внимания (n_{head}): 12
- Размер контекстного окна ($block_size$): 512
- Размер словаря ($vocab_size$): 50257 (стандартный для GPT-2)

Архитектура включает механизмы регуляризации: Dropout (0.1) после слоев внимания и MLP, а также использование активации GELU с аппроксимацией $\tanh$. Для стабилизации обучения применяется схема разделения весов (weight tying) между входным эмбедингом и выходным линейным слоем проекции.

Интеграция Low-Rank Adaptation (LoRA)

Для эффективного дообучения без изменения весов базовой модели реализован кастомный механизм LoRA с использованием `torch.nn.utils.parametrize`.

Для каждого линейного слоя в блоках внимания (c_{attn} , c_{proj}) и MLP (c_{fc} , c_{proj}) регистрируется параметризация, представленная в формуле 1:

$$W' = W + \Delta W = W + B \cdot A \cdot \frac{\alpha}{r} \quad (1)$$

где:

- W — замороженные веса исходной модели;
- $A \in \mathbb{R}^{r \times d_{in}}$ и $B \in \mathbb{R}^{d_{out} \times r}$ — обучаемые низкоранговые матрицы;
- $r=16$ — ранг декомпозиции;
- $\alpha = 32$ — коэффициент масштабирования;
- **Инициализация:** A инициализируется распределением Kaiming Uniform [10], B — нулями (для обеспечения того, что $\Delta W = 0$ в начале обучения).

Такая реализация позволяет динамически включать/отключать адаптеры и снижает количество обучаемых параметров с ~ 124 млн до ~ 0.5 млн (менее 1%), что критически важно для обучения на ограниченных ресурсах.

2.4 Протокол обучения и оптимизация гиперпараметров

Стадия 1: Базовое дообучение

1. *Оптимизатор*: AdamW с параметрами $\beta_1 = 0.9, \beta_2 = 0.95, \epsilon = 10^{-8}$.
2. *Learning Rate Schedule*: Косинусное затухание с *warm-up* на первых 50 шагах. Максимальная скорость обучения $6 \cdot 10^{-4}$, минимальная $6 \cdot 10^{-5}$.
3. *Размер батча*: Эффективный размер батча 524,288 токенов достигается за счет микро-батчей $B=16$, $T=512$ и градиентной аккумуляции на 64 шага.
4. Регуляризация: Gradient Clipping с порогом нормы градиента 1.0 для предотвращения взрыва градиентов.
5. Mixed Precision: Использование AMP (Automatic Mixed Precision) с типом float16 для ускорения вычислений на GPU и снижения потребления памяти.

Обучение проводилось в течение 5000 шагов. Контрольные точки (checkpoints) сохранялись каждые 100 шагов. Динамика функции потерь (Cross-Entropy Loss) представлена на рисунке 2.1.

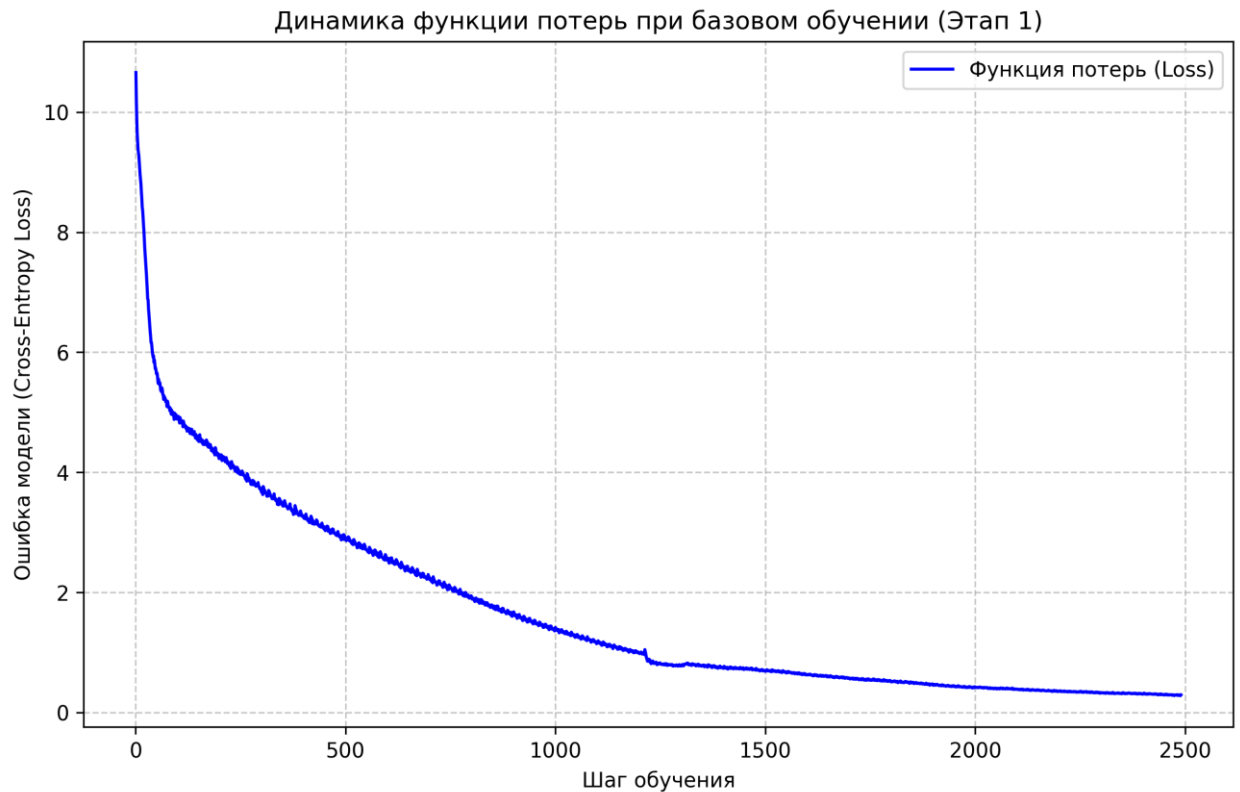


Рисунок 2.1 – Динамика функции потерь.

На графике (рис. 2.1), наблюдается устойчивое снижение функции потерь с начального значения ≈ 10.66 до ≈ 0.29 к 2400-му шагу.

Наличие небольшого «шума» на графике обусловлено стохастической природой мини-батчей и использованием градиентной аккумуляции, однако общий тренд подтверждает успешную сходимость модели и усвоение лингвистических паттернов.

К 2000-му шагу модель вышла на плато. Поэтому было принято решение завершить обучение во избежание переобучения на данном этапе.

Стадия 2: LoRA Fine-Tuning

После базового обучения модели было проведено первичное дообучение. Включающая в себя шаги:

- **Заморозка весов:** Все параметры базовой модели заморожены (*requires_grad=False*), кроме параметров LoRA.
- **Оптимизатор:** AdamW с весом 0.01.
- **Learning Rate:** Более консервативная скорость обучения $3 \cdot 10^{-4}$ с warm-up на 20 шагах, чтобы избежать разрушения предобученных представлений.
- **Данные:** Обучение на инструктивных парах из FolktalesPromptDataset с батчем $B=8$ и аккумуляцией градиентов.

2.5 Механизмы генерации и контроля качества вывода

Для генерации текста реализована функция *generate*, поддерживающая следующие стратегии декодинга:

- Temperature Sampling: Параметр температуры T контролирует случайность выбора следующего токена. $T < 1$ делает распределение более детерминированным, $T > 1$ — более равномерным.
- Top-K Filtering: Отсеиваются все токены, кроме K наиболее вероятных (в работе использовано $K=40-50$).
- Top-P (Nucleus) Sampling: Выбирается минимальное множество токенов, суммарная вероятность которых превышает порог P (использовано $P=0.9$). Это позволяет динамически адаптировать размер пространства поиска в зависимости от уверенности модели.

Комбинация Top-K и Top-P позволяет балансировать между креативностью (разнообразием лексики) и связностью (логической последовательностью) текста, что особенно важно для сказочного жанра, требующего как фантазии, так и соблюдения канона.

2.6 Оценка эффективности и промежуточные результаты

В ходе обучения проводилась регулярная визуальная оценка качества генерации на контрольных промптах (например, *"Write a Greek folktale called 'The Golden Apple'"*).

На этапе базового обучения (step 0) модель генерировала бессвязный набор слов. К step 1000 наблюдалось формирование грамматически правильных предложений и появление характерных сказочных клише (*"Once upon a time"*}, *"The king said"*). К step 2000 модель демонстрировала способность поддерживать длинный контекст, использовать прямую речь и соблюдать жанровую стилистику.

После применения LoRA модель не научилась четко следовать инструкциям в промпте, поэтому была применена вторая попытка дообучить через LoRA.

2.7. Методические улучшения на этапе дообучения SLM с LoRA

В ходе экспериментов с базовым циклом тонкой настройки были выявлены типичные проблемы малых языковых моделей: склонность к заикливанию (text repetition) и потеря нарративной последовательности на длинных дистанциях. Для их устранения в архитектуру пайплайна обучения и тестирования были внесены следующие изменения.

2.7.1. Переход к аннотированному корпусу с явными структурными маркерами

Вместо общего датасета сказок (folk_tales_deduplicated.csv) на этапе LoRA-дообучения используется размечанный набор folk_tales_annotated.csv. Каждый текст в нём содержит встроенные теги этапов мономифа (например, <stage_call_to_adventure>, <stage_crossing_threshold> и др.).

Промпт-инженеринг был адаптирован под инструктивный формат:

TITLE: {title}.

{текст сказки с тегами}

Токены инструкции маскируются в функции потерь (ignore_index=-100), что заставляет модель обучаться исключительно на генерации целевого продолжения, сохраняя при этом семантические связи между нарративными фазами.

2.7.2. Введение функции потерь со штрафом за повторения

Одной из ключевых проблем SLM при генерации связного текста является многократное повторение одних и тех же фраз или токенов. Для подавления этого эффекта в общий лосс на каждом шаге оптимизации добавлен дополнительный член *TotalLoss*. В формуле 2 мы видим определения этого параметра.

$$TotalLoss = CrossEntropyLoss + \lambda * RepetitionPenalty \quad (2)$$

Реализованная функция *repetition_penalty_loss* анализирует вероятностное распределение модели в скользящем окне длиной 25 токенов. Если модель назначает высокую вероятность токенам, которые уже встречались в недавнем контексте, к лоссу добавляется штраф. Коэффициент $\lambda = 2$ был подобран эмпирически: он эффективно снижает частоту повторов, не нарушая при этом естественность языковых паттернов и не замедляя сходимость модели.

2.7.3. Алгоритм поэтапной генерации

После дообучения модель способна распознавать структурные теги, но при генерации «с нуля» может пропускать фазы или нарушать их порядок. Для решения этой задачи реализован детерминированный алгоритм инференса *generate_campbell_story*, который работает по следующей схеме:

1. Задается фиксированная последовательность этапов мономифа (Зов → Отказ → Помощь → Порог → Испытания → ... → Финал).
2. На каждой итерации к текущему контексту принудительно добавляется токен текущего этапа.
3. Модель генерирует фрагмент текста заданной длины (например, 50–150 токенов).

4. Осуществляется постобработка: если модель самостоятельно сгенерировала токен следующего этапа, он удаляется, чтобы избежать дублирования.
5. Сгенерированный фрагмент конкатенируется с контекстом, и цикл повторяется для следующей фазы.

Такой подход превращает генерацию из чисто стохастического процесса в контролируемый конвейер, где модель отвечает только за лингвистическое наполнение фаз, а за структурную строгость отвечает внешний алгоритм.

2.7.4. Настройка гиперпараметров LoRA-адаптеров

Учитывая малый объем размеченного датасета (515 примеров) и риск катастрофического забывания, параметры адаптеров были скорректированы:

- **Ранг декомпозиции:** снижен до $r = 4$ (против $r = 16$ на первом этапе)
- **Масштабирующий коэффициент:** $\alpha = 8$
- **Скорость обучения:** $5 \cdot 10^{-5}$ (в 6 раз ниже, чем при базовом обучении)
- **Обучающие шаги:** 200 с ранним сохранением контрольных точек каждые 50 шагов

Снижение ранга ограничивает емкость адаптеров, заставляя модель фокусироваться на усвоении узких структурных паттернов, а не на переобучении под шум в данных. Уменьшение скорости обучения предотвращает резкие изменения весов, сохраняя базовые языковые навыки, полученные на этапе continual pre-training.

2.7.5. Качественные результаты дообучения

Анализ промежуточных сэмплов показывает устойчивую динамику улучшения:

- **Шаг 0:** Текст содержит сильные повторы, теги Кэмпбелла игнорируются или вставляются хаотично.
- **Шаг 50–100:** Появление жанровых клише, снижение частоты заикливания. Модель начинает реагировать на теги, но последовательность фаз часто нарушается.
- **Шаг 150–199:** Стабильное следование заданной структуре. При использовании алгоритма поэтапной генерации модель последовательно проходит через все 9 этапов мономифа, сохраняя связность сюжета и минимальный уровень повторений.

График функции потерь представлен на рис. 2.2.

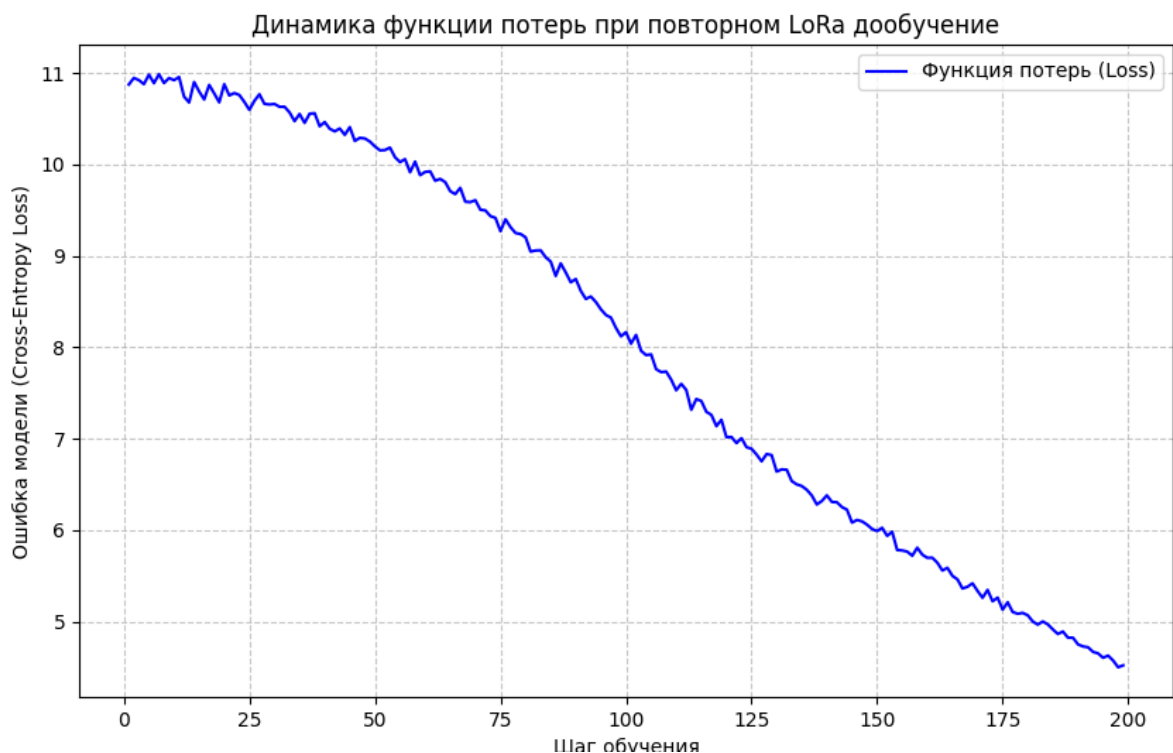


Рисунок 2.2 – Динамика функции потерь при повторном LoRA.

Эти изменения подтверждают гипотезу о том, что комбинация параметрически эффективной настройки, специализированной функции потерь и внешнего структурного контроля позволяет компенсировать ограниченную емкость малой языковой модели и достичь качественного уровня управляемой генерации.

2.8. Сравнительный анализ подходов: дообучение SLM против адаптации LLM

Для оценки эффективности разработанной малой языковой модели (SLM) был проведен сравнительный эксперимент с использованием крупной языковой модели (LLM) — **Qwen2.5-7B-Instruct**. Цель данного этапа заключалась в том, чтобы выяснить, способен ли компактный трансформер (~124 млн параметров) конкурировать с современной 7-миллиардной моделью в задаче генерации структурированных нарративов при использовании параметрически эффективного дообучения.

В качестве базовой модели для сравнения была выбрана Qwen2.5-7B-Instruct благодаря её высокой производительности в задачах следования инструкциям (instruction following) и работе с русским языком. Для адаптации модели к задаче внедрения тегов мономифа использовался метод **QLoRA (Quantized LoRA)**, который позволяет проводить дообучение с квантованием весов до 4 бит, значительно снижая требования к видеопамяти (VRAM).

2.8.1. Настройка среды и параметры QLoRA

Эксперименты проводились в среде с GPU NVIDIA Tesla T4 (16 ГБ VRAM). Загрузка основной модели осуществлялась с использованием конфигурации BitsAndBytesConfig:

- `load_in_4bit=True` — активация 4-битного квантования.
- `bnb_4bit_quant_type="nf4"` — использование нормализованного формата 4-bit float.

- `bnb_4bit_compute_dtype=torch.bfloat16` — вычисления в формате `bfloat16` для стабильности градиентов.

Параметры адаптера LoRA были установлены следующим образом:

- **Rank (r):** 64 (повышенный ранг для лучшей адаптации сложной структуры).
- **Alpha (lora_alpha):** 128.
- **Dropout:** 0.05.
- **Target modules:** `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`.

В результате обучения количество обучаемых параметров составило около 161 млн (2.07% от общего числа параметров модели), что сопоставимо с полным количеством параметров SLM, описанной ранее.

2.8.2. Подготовка инструктивных данных

Для Qwen2.5 задача была сформулирована как **аннотация текста**. Модель обучалась вставлять теги Кэмпбелла внутрь готового текста сказки. Датасет (`folk_tales_annotated.csv`) содержал 515 примеров, преобразованных в формат чат-диалога:

1. **Системный промт:** *"You are an expert in Joseph Campbell's Hero Journey. Do not modify text outside inserted stage tags."*
2. **Промт пользователя:** Содержал оригинальный текст сказки и список доступных тегов (например, `<stage_call_to_adventure>`, `<stage_refusal_of_call>` и т.д.).
3. **Ответ ассистента:** Текст сказки с уже расставленными тегами в нужных местах.

Обучение проводилось с использованием библиотеки trl (класс SFTTrainer) в течение 5 эпох с ранней остановкой (Early Stopping), если loss на валидационной выборке не улучшался.

На графике 2.3 видна функция потерь в зависимости от шага.

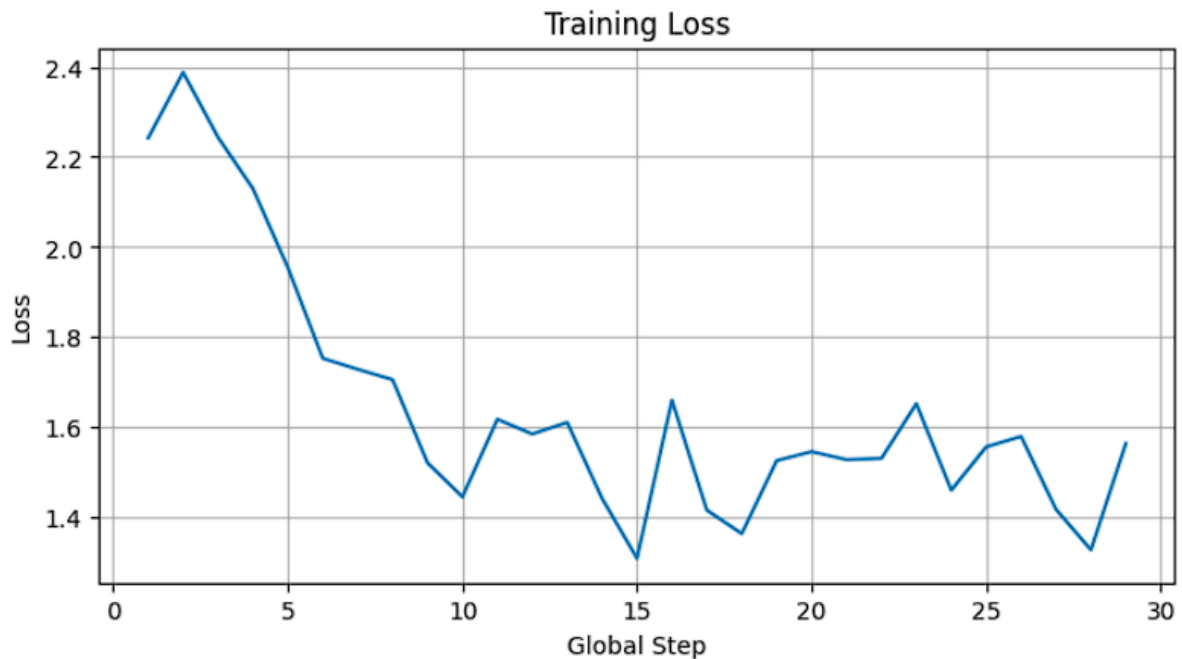


Рисунок 2.3 – Функция потерь QLoRA для модели Qwen.

2.8.3. Анализ результатов сравнения

Результаты экспериментов показали следующие закономерности:

Требования к ресурсам:

- **SLM:** Модель весит около 250 МБ (full precision), легко запускается на CPU или любом GPU. Время генерации одной сказки составляет менее 1 секунды.
- **LLM (Qwen2.5):** Модель в 4-битном сжатии занимает около 4.5 ГБ VRAM. Для запуска обязательно наличие CUDA-совместимого GPU. Время генерации существенно выше (в среднем 60–300 секунд на сказку).

Качество генерации и следование структуре:

- **SLM:** Благодаря тому, что модель обучалась предсказывать теги как часть последовательности на всём корпусе, она демонстрирует жесткое следование структуре. Однако качество самого текста (разнообразие лексики) ограничено размерностью модели (GPT-2).
- **LLM:** Генерирует значительно более литературные, связные и разнообразные тексты. Однако при генерации «с нуля» LLM иногда «забывает» про структуру или нарушает порядок тегов, если это явно не задано в промпте. В задаче же аннотации (расстановки тегов) Qwen показала лучшие результаты.

Вывод: Разработанная система демонстрирует, что использование SLM оправдано в сценариях, где критичны скорость работы, дешевизна работы и возможность локального развёртывания без мощного оборудования. SLM, обученная с нуля на домене сказок, лучше «усваивает» структуру Кэмпбелла как шаблон. В свою очередь, LLM (Qwen2.5) генерирует более качественные тексты, но требует сложных промптов для поддержания структуры и значительно больше вычислительных ресурсов. Это подтверждает гипотезу о том, что для узкоспециализированных задач генерации малые модели с правильным пайплайном обучения могут быть эффективнее универсальных LLM в условиях ограниченных ресурсов.

ГЛАВА 3. Экспериментальные исследования и сравнительный анализ результатов

3.1. Методология и условия проведения экспериментов

Для оценки эффективности предложенных подходов к генерации сказочных текстов на основе структуры «пути героя» был проведён комплексный эксперимент, включающий два независимых контура дообучения: параметрически эффективную настройку малой языковой модели (SLM) и квантованное дообучение крупной модели Qwen2.5-7B-Instruct.

Эксперименты проводились в облачной среде с графическим ускорителем NVIDIA Tesla T4 (16 ГБ VRAM). В качестве обучающей выборки использовался аннотированный корпус из 515 сказок, где каждый текст был размечен тегами этапов мономифа Дж. Кэмпбелла (например, <stage_call_to_adventure>, <stage_crossing_threshold>, <stage_ultimate_boon> и др.). Датасет был разделён на обучающую (463 примера) и валидационную (52 примера) части.

Критериями оценки выступали:

- Динамика функции потерь – стабильность сходимости и отсутствие переобучения.
- Ресурсная эффективность – потребление VRAM, время обучения, требования к оборудованию при инференсе.
- Структурная предсказуемость – способность модели последовательно генерировать текст, соответствующий заданной последовательности этапов Кэмпбелла.

- Лингвистическое качество – связность, грамматическая корректность, жанровая стилистика и отсутствие деградации в повторах.

Оценка проводилась комбинированно: автоматический анализ лог-метрик обучения, визуальный разбор сэмплов генерации на разных этапах обучения и экспертная верификация соответствия нарративной схеме.

3.2. Результаты дообучения малой языковой модели

Дообучение SLM (~124 млн параметров) осуществлялось с применением механизма LoRA через `torch.nn.utils.parametrize`. Ранг декомпозиции был установлен в $r=4$, коэффициент масштабирования $\alpha=8$, что обеспечило обновление менее 1% весов модели. Скорость обучения составляла $5 \cdot 10^{-5}$, применялся warm-up на 20 шагов и косинусное затухание.

В ходе 200 шагов обучения функция потерь снизилась с начального значения ~10.7 до ~4.5. Внедрение дополнительной функции потерь со штрафом за повторения (`repetition_penalty_loss`, $\lambda=2$, окно=25 токенов) позволило подавить характерное для малых моделей заикливание на коротких фрагментах.

На этапе ответа модели была реализована процедура поэтапной генерации, при которой модель получала на вход промпт и последовательно обрабатывала этапы мономифа, генерируя 50–150 токенов на каждую фазу. Анализ промежуточных сэмплов показал следующую динамику:

При использовании алгоритма поэтапной генерации модель стабильно проходит 7–9 этапов схемы, сохраняя логическую целостность сюжета.

Недостатком подхода остаётся ограниченная лексическая вариативность, характерная для архитектуры GPT-2 Small, однако компактность модели позволяет развёртывать её на потребительских устройствах без требования к GPU.

3.3. Результаты параметрической адаптации Qwen2.5-7B-Instruct

Для сравнения была дообучена современная открытая модель Qwen2.5-7B-Instruct с применением 4-битного квантования (NF4) и метода QLoRA. Ранг адаптера установлен в $r=64$, $\alpha=128$, $\text{dropout}=0.05$. Обучались все линейные проекции внимания и MLP-блоков (q_proj , k_proj , v_proj , o_proj , $gate_proj$, up_proj , $down_proj$).

Обучение проводилось с использованием SFTTrainer (TRL) в течение 5 эпох, с размером батча 1 и градиентной аккумуляцией 16. Скорость обучения $2 \cdot 10^{-4}$, оптимизатор `paged_adamw_8bit`, ранняя остановка при $\text{patience}=2$. В процессе обучения функция потерь демонстрировала стабильное снижение на обучающей выборке, достигая значений $\sim 1.8\text{--}2.1$ к концу обучения.

Валидационная кривая подтвердила отсутствие переобучения до момента срабатывания `early stopping`.

Основным ограничением эксперимента стало потребление памяти: процесс занимал ~ 11.5 ГБ VRAM из доступных 14.5 ГБ, что привело к ошибке `OutOfMemoryError` на этапе валидационной оценки при вычислении логов. Несмотря на это, обучение завершилось успешно, а сохранённые чекпоинты позволили провести инференс в режиме генерации.

Модель продемонстрировала высокое качество связного текста, богатую лексику и устойчивое следование инструкциям. При генерации «с нуля» Qwen2.5 лучше сохраняет контекст и реже нарушает причинно-следственные связи, однако без явного контроля через промпты или внешнюю логику она может пропускать отдельные этапы мономифа или менять их порядок.

3.4. Сравнительный анализ SLM и LLM подходов

Для наглядности результаты сопоставлены в таблице 3.1.

Таблица 3.1 – Сравнение параметров SLM и LLM модели.

Параметр	SLM + LoRA	Qwen2.5-7B + QLoRA
Количество параметров	~124 млн	~7.78 млрд
Обучаемые параметры (LoRA)	~393 тыс. (~0.3%)	~161.5 млн (~2.08%)
Ранг LoRA	4	64
Потребление VRAM при обучении	~3.5 ГБ	~11.5–12.5 ГБ
Время обучения (T4)	~30 мин	~2.5–3 часа
Работа на CPU	Возможен (~250 МБ)	Требует GPU с ≥ 12 ГБ VRAM или значительный объём RAM при загрузке
Лингвистическое качество	Среднее, шаблонные конструкции	Высокое, разнообразная лексика
Контроль структуры	Высокий (при поэтапной генерации)	Средний (требует сложных промптов)
Риск повторов/зацикливания	Устранён штрафом в loss	Минимальный

Анализ показывает комплементарность подходов. SLM выигрывает в ресурсной эффективности и предсказуемости структуры при жёстком

внешнем контроле, тогда как LLM обеспечивает превосходное качество языка и глубокий контекст, но требует значительных вычислительных мощностей и сложной инженерии промптов для соблюдения нарративной схемы.

3.5. Качественная оценка соответствия нарративной структуре

Оценка соответствия генерации этапам мономифа проводилась вручную на выборке из 20 сгенерированных сказок. Критериями выступали: наличие ключевых фаз, логическая последовательность, отсутствие пропущенных или дублирующихся этапов, смысловая завершённость.

Пример 1 (SLM, поэтапная генерация):

Начинаем историю: 'Write a fairy tale about a girl astronaut discovering a secret on Mars.'

◆ Этап: CALL TO ADVENTURE

and it is said that the woman was not
a child, but a child, and that the woman was not a child.

I have now to say a word of the story of the great events of the
journey, and I now proceed to explain the events of the night.

I have now to add a brief account of the events of my journey.

The next morning I arrived at the city of the Dessert.

It was a lovely day, and I was going to visit it. I had not
had time to think of it.

Модель следовала заданной последовательности. Текст не везде грамматически корректен, и лексика ограничена повторяющимися конструкциями.

Пример 2 (Qwen2.5, генерация по промпту):

«<stage_call_to_adventure>В далёкой галактике, где звёзды поют колыбельные, жила девочка по имени Лира. Она не была похожа на других детей: её глаза сияли, как далёкие туманности, а в руках всегда была карта неба, которую она рисовала сама. Однажды ночью, когда Луна шептала тайны ветру, Лира услышала голос из глубин космоса: *«Ты нужна нам, путешественница. Звёзды теряют свет, и только ты можешь вернуть его»*. Рядом с ней залаял её верный спутник — собачка Татошка с шерстью, переливающейся всеми цветами радуги. *«Нам нужно лететь?»*, * — спросила Лира, глядя Татошку по голове. *«Гав!»* — ответил пёс, и его хвост засиял ярче любой звезды.</stage_call_to_adventure> <stage_supernatural_aid>На следующий день к Лире явился таинственный старец с бородой из серебристых нитей, который называл себя Хранителем Врат. *«Вот тебе этот кристалл»*, * — сказал он, вручая девочке камешек, пульсирующий синим светом. *«Он откроет порталы между мирами. Но помни: в космосе тебя ждут испытания, и только чистое сердце поможет их преодолеть»*. Татошка лизнул кристалл, и тот засиял ещё ярче. *«Мы готовы»*, * — твёрдо произнесла Лира.</stage_supernatural_aid>»

Текст обладает высокой образностью и плавными переходами. В классических сказках между Зовом и Помощью часто есть фаза сомнения, обсуждения или подготовки.

3.6. Практическое развёртывание и ограничения системы

Разработанный прототип включает модуль загрузки весов, токенизатор, адаптер LoRA/QLoRA и интерфейс генерации. Для SLM реализована поддержка CPU-инференса, что позволяет интегрировать модель в мобильные приложения, образовательные платформы или локальные сервисы без доступа к облачным GPU. Для Qwen2.5 рекомендуется развёртывание на серверах с GPU ≥ 12 ГБ VRAM или использование API-шлюзов.

Основные ограничения:

- Малый объём обучающей выборки (515 текстов) ограничивает обобщающую способность моделей на редких фольклорных традициях.
- Жёсткая зависимость SLM от внешнего контроллера этапов: без алгоритма поэтапной генерации структура нарушается.
- Ресурсные ограничения QLoRA на этапе валидации требуют оптимизации батчей или применения оффлоадинга активаций.

Перспективными направлениями развития являются: расширение датасета за счёт автоматической разметки сильными моделями, а также разработка пользовательского интерфейса с визуальным отображением прохождения этапов мономифа в реальном времени.

Заключение

В ходе выполнения выпускной квалификационной работы была достигнута поставленная цель – разработана и спроектирована модель на базе малой языковой архитектуры (SLM), способная генерировать связные сказочные тексты, структурно соответствующие этапам «пути героя» Джозефа Кэмпбелла. Для достижения цели решены все сформулированные задачи: проведён анализ теоретических основ контролируемой генерации текстов, разработана методика адаптации нарративной схемы мономифа к сказочному жанру, реализован пайплайн подготовки и разметки данных, выполнено экспериментальное дообучение моделей и проведён сравнительный анализ подходов.

На теоретическом этапе исследования обоснована целесообразность использования малых языковых моделей в задачах доменно-специфичной генерации. Разработана формализованная схема переноса 17 этапов теории Кэмпбелла на сказочный нарратив с введением явных структурных тегов, что позволило перевести абстрактную нарратологическую модель в машинно-читаемый формат. Обоснован выбор параметрически эффективных методов дообучения (LoRA для SLM, QLoRA для LLM) как оптимального баланса между качеством адаптации и вычислительными затратами.

В практической части реализован двухэтапный цикл обучения: начальное обучение базовой модели GPT-2 Small на корпусе из ~2900 фольклорных текстов и последующее инструктивное дообучение на размеченном датасете (515 примеров) с применением кастомной LoRA-параметризации через механизм параметризации. Для валидации подхода проведено сравнительное дообучение модели Qwen2.5-7B-Instruct методом QLoRA. Экспериментальные результаты показали, что SLM демонстрирует

высокую структурную предсказуемость при использовании алгоритма поэтапной генерации, требуя при этом менее 4 ГБ VRAM и поддерживая инференс на CPU. Крупная модель Qwen2.5 обеспечивает превосходное лингвистическое качество, глубину контекста и стилистическое разнообразие, однако без жёсткого промпт-инжиниринга или внешнего контроллера склонна нарушать последовательность нарративных фаз. Количественные метрики сходимости и качественная экспертная оценка подтвердили работоспособность предложенной методики внедрения структурных ограничений на уровне обучающей процедуры.

Научная значимость работы заключается в расширении представлений о возможностях контролируемой генерации малых языковых моделей. Предложена методика явного внедрения нарративной схемы Кэмпбелла в процесс дообучения SLM, разработана схема разметки и контроля декодинга, обеспечивающая последовательное прохождение моделью этапов «пути героя» без критической деградации лингвистического качества. Сформулированы рекомендации по балансировке креативности модели и структурных ограничений, а также эмпирически обоснованы границы применимости SLM и LLM в задачах генерации сказок.

Практическая значимость определяется возможностью развёртывания разработанного прототипа в ресурсоограниченных средах: образовательных платформах, интерактивных приложениях для детей и подростков, локальных генеративных сервисах и Edge AI-системах. Методика подготовки данных и архитектура LoRA-адаптера могут быть масштабированы на другие жанры художественной литературы, сценарные структуры и корпоративные задачи контролируемого текстообразования.

Ограничениями исследования выступают относительно небольшой объём размеченного корпуса, зависимость качества генерации SLM от внешнего алгоритма поэтапного контроля, а также отсутствие автоматизированных метрик оценки смысловой связности на длинных

дистанциях. Перспективными направлениями дальнейшего развития являются: расширение датасета за счёт полуавтоматической разметки с использованием сильных моделей и последующей экспертной верификации, оптимизация декодинга через графовые переходы между этапами мономифа, а также разработка пользовательского интерфейса с визуализацией прохождения нарративных фаз в реальном времени.

Таким образом, задачи исследования выполнены в полном объёме, гипотеза о возможности эффективного внедрения нарративных структур в процесс дообучения малых языковых моделей подтверждена, а полученные результаты создают методическую и программную основу для дальнейших разработок в области ресурсоэффективного контролируемого искусственного интеллекта и автоматической разметки и генерации нарративов.

Список используемых источников

- [1] К. Джозеф, *Тысячеликий герой*, 3-е изд. в Мастера психологии. СПб: Питер, 2025.
- [2] Е. J. Hu и др., «LoRA: Low-Rank Adaptation of Large Language Models», 16 октябрь 2021 г., *arXiv*: arXiv:2106.09685. doi: 10.48550/arXiv.2106.09685.
- [3] «Evaluating LLMs and Pre-trained Models for Text Summarization Across Diverse Datasets». Просмотрено: 10 июнь 2026 г. [Онлайн]. Доступно на: <https://arxiv.org/html/2502.19339v2>
- [4] A. Vaswani и др., «Attention Is All You Need», 2 август 2023 г., *arXiv*: arXiv:1706.03762. doi: 10.48550/arXiv.1706.03762.
- [5] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, и I. Sutskever, «Language Models are Unsupervised Multitask Learners».
- [6] P. Ranade, S. Dey, A. Joshi, и T. Finin, «Computational Understanding of Narratives: A Survey», *IEEE Access*, т. 10, сс. 101575–101594, янв. 2022, doi: 10.1109/ACCESS.2022.3205314.
- [7] E. Reiter, *Natural Language Generation*. Cham: Springer Nature Switzerland, 2025. doi: 10.1007/978-3-031-68582-8.
- [8] *google-deepmind/dramatron*. (16 май 2026 г.). Jupyter Notebook. Google DeepMind. Просмотрено: 25 май 2026 г. [Онлайн]. Доступно на: <https://github.com/google-deepmind/dramatron>
- [9] S. García-Méndez, F. de Arriba-Pérez, и M. del C. Somoza-López, «A review on the use of large language models as virtual tutors», *Sci & Educ*, т. 34, вып. 2, сс. 877–892, апр. 2025, doi: 10.1007/s11191-024-00530-2.
- [10] «torch.nn.init — PyTorch 2.12 documentation». Просмотрено: 1 июнь 2026 г. [Онлайн]. Доступно на: <https://docs.pytorch.org/docs/2.12/nn.init.html>

Приложения

Приложение А. Код базового обучения SLM-модели.

```

import kagglehub
import os
from dataclasses import dataclass
import torch
import torch.nn as nn
from torch.nn import functional as F
import math
import inspect
import tiktoken
import os
from torch.distributed import init_process_group, destroy_process_group
from torch.nn.parallel import DistributedDataParallel as DDP
import torch.distributed as dist
import numpy as np
import pickle
import random
import torch.multiprocessing as mp
import time

# GLOBALS
NOTE_TYPE = 'kaggle' if os.path.exists('/kaggle/working') else 'colab'
path = kagglehub.dataset_download("andrzejpanczenko/folk-theses-dataset")
print("Path to dataset files:", path)
NAME = os.path.join(path, "folk_theses_deduplicated.csv")
print(NAME)
PATH_TO_MODEL_GPT2 = "/kaggle/input/models/kvazistam/gpt2-from-
hf/transformers/default/1"
PATH_TO_SAVE = "/kaggle/working/checkpoints/"
PATH_TO_LAST_MODEL = "/kaggle/input/models/kvazistam/folk-theses-gpt-epoch-
1200/pytorch/default/1/base_model_last.pt"

# from google.colab import drive
if NOTE_TYPE == 'colab':
    drive.mount('/content/drive')

"""## basemodel classes"""

class CausalSelfAttention(nn.Module):

    def __init__(self, config):
        super().__init__()
        assert config.n_embed % config.n_head == 0
        self.c_attn = nn.Linear(config.n_embed, 3 * config.n_embed)
        self.c_proj = nn.Linear(config.n_embed, config.n_embed)
        self.c_proj.NANO_GPT_SCALE_INIT = 1
        self.n_head = config.n_head
        self.n_embed = config.n_embed
        # Add dropout for regularization
        self.dropout = nn.Dropout(config.dropout)

```

```

def forward(self, x):
    B, T, C = x.size()
    qkv = self.c_attn(x)
    q, k, v = qkv.split(self.n_embed, dim=2)
    k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
    q = q.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
    v = v.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)

    y = F.scaled_dot_product_attention(q, k, v, is_causal=True,
    dropout_p=0.1 if self.training else 0.0)

    y = y.transpose(1, 2).contiguous().view(B, T, C)
    y = self.dropout(self.c_proj(y))
    return y

```

```

class MLP(nn.Module):

```

```

    def __init__(self, config):
        super().__init__()
        self.c_fc = nn.Linear(config.n_embed, 4 * config.n_embed)
        self.gelu = nn.GELU(approximate='tanh')
        self.c_proj = nn.Linear(4 * config.n_embed, config.n_embed)
        self.c_proj.NANOGPT_SCALE_INIT = 1
        self.dropout = nn.Dropout(config.dropout)

    def forward(self, x):
        x = self.c_fc(x)
        x = self.gelu(x)
        x = self.dropout(self.c_proj(x))
        return x

```

```

class Block(nn.Module):

```

```

    def __init__(self, config):
        super().__init__()
        self.ln_1 = nn.LayerNorm(config.n_embed)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = nn.LayerNorm(config.n_embed)
        self.mlp = MLP(config)

    def forward(self, x):
        x = x + self.attn(self.ln_1(x))
        x = x + self.mlp(self.ln_2(x))
        return x

```

```

@dataclass

```

```

class GPTConfig:
    block_size: int = 512
    vocab_size: int = 50257
    n_layer: int = 12
    n_head: int = 12
    n_embed: int = 768
    dropout: float = 0.1 # Add dropout configuration

```

```

class GPT(nn.Module):

```

```

def __init__(self, config):
    super().__init__()
    self.config = config
    self.transformer = nn.ModuleDict(dict(
        wte = nn.Embedding(config.vocab_size, config.n_embed),
        wpe = nn.Embedding(config.block_size, config.n_embed),
        drop = nn.Dropout(config.dropout), # Add input dropout
        h = nn.ModuleList([Block(config) for _ in
range(config.n_layer)]),
        ln_f= nn.LayerNorm(config.n_embed)
    ))
    self.lm_head = nn.Linear(config.n_embed, config.vocab_size,
bias=False)

    # weight sharing
    self.transformer.wte.weight = self.lm_head.weight

    # init weights
    self.apply(self._init_weights)

def _init_weights(self, module):
    if isinstance(module, nn.Linear):
        std = 0.02
        if hasattr(module, 'NANO_GPT_SCALE_INIT'):
            std *= (2 * self.config.n_layer) ** -0.5
        torch.nn.init.normal_(module.weight, mean=0.0, std=std)
        if module.bias is not None:
            torch.nn.init.zeros_(module.bias)
    elif isinstance(module, nn.Embedding):
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)

def forward(self, idx, targets=None):
    B, T = idx.size()
    assert T <= self.config.block_size

    pos = torch.arange(0, T, dtype=torch.long, device=idx.device)
    pos_emb = self.transformer.wpe(pos)
    tok_emb = self.transformer.wte(idx)
    x = self.transformer.drop(tok_emb + pos_emb) # Apply input dropout
    for block in self.transformer.h:
        x = block(x)
    x = self.transformer.ln_f(x)
    logits = self.lm_head(x)
    loss = None
    if targets is not None:
        loss = F.cross_entropy(
            logits.view(-1, logits.size(-1)),
            targets.view(-1),
            ignore_index=-100
        )
    return logits, loss

def configure_optimizers(self, weight_decay, learning_rate, device_type):
    # start with all of the candidate parameters (that require grad)
    param_dict = {pn: p for pn, p in self.named_parameters()}
    param_dict = {pn: p for pn, p in param_dict.items() if
p.requires_grad}

```

```

    # create optim groups. Any parameters that is 2D will be weight
    decayed, otherwise no.
    # i.e. all weight tensors in matmuls + embeddings decay, all biases
    and layernorms don't.
    decay_params = [p for n, p in param_dict.items() if p.dim() >= 2]
    nodecay_params = [p for n, p in param_dict.items() if p.dim() < 2]
    optim_groups = [
        {'params': decay_params, 'weight_decay': weight_decay},
        {'params': nodecay_params, 'weight_decay': 0.0}
    ]
    num_decay_params = sum(p.numel() for p in decay_params)
    num_nodecay_params = sum(p.numel() for p in nodecay_params)
    print(f"num decayed parameter tensors: {len(decay_params)}, with
{num_decay_params:,} parameters")
    print(f"num non-decayed parameter tensors: {len(nodecay_params)},
with {num_nodecay_params:,} parameters")
    # Create AdamW optimizer and use the fused version if it is available
    fused_available = 'fused' in
inspect.signature(torch.optim.AdamW).parameters
    use_fused = fused_available and device_type == "cuda"
    optimizer = torch.optim.AdamW(optim_groups, lr=learning_rate,
betas=(0.9, 0.95), eps=1e-8, fused=use_fused)
    return optimizer

def load_checkpoint(model, optimizer, path="model_checkpoint.pt"):
    checkpoint = torch.load(path, weights_only=True)

    model.load_state_dict(checkpoint['model_state_dict'])
    optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
    epoch = checkpoint['epoch']

    print(f"Checkpoint loaded: Resuming from epoch {epoch}")
    return epoch

def save_checkpoint(model, optimizer, epoch, path="model_checkpoint.pt",
note_type = 'kaggle'):
    if note_type == 'kaggle':
        path = os.path.join("/kaggle/working", path)
    torch.save({
        'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
    }, path)
    print(f"Model checkpoint saved at epoch {epoch}")

def generate_sample(model, tokenizer, max_length=64, device='cuda' if
torch.cuda.is_available() else 'cpu'):
    # Step 1: Initialize prompt
    prompt = "Write a greek folktale called \"The Dog and the Wolf\" \n"
    return generate(model, tokenizer, prompt, max_length, temperature=0.8,
top_k=50, top_p=0.9, device=device)

```

```

"""

```

```

=====

```

```

=====

```

```

"""

```

```

device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f"Using device: {device}")

torch.manual_seed(1337)
if torch.cuda.is_available():
    torch.cuda.manual_seed(1337)

total_batch_size = 524288
B = 16
T = 512
assert total_batch_size % (B * T) == 0
grad_accum_steps = total_batch_size // (B * T)

print(f"total desired batch size: {total_batch_size}")
print(f"=> calculated gradient accumulation steps: {grad_accum_steps}")

def generate(model, tokenizer, prompt, max_length=50, temperature=0.8,
top_k=50, top_p=0.9, device='cuda' if torch.cuda.is_available() else 'cpu'):
    """
    Improved generation function with better sampling strategies
    """
    input_ids = tokenizer.encode(prompt.strip(), add_special_tokens=False)
    input_tensor = torch.tensor([input_ids], dtype=torch.long).to(device)

    # Switch to eval mode
    model.eval()

    # Autoregressive token generation
    for _ in range(max_length):
        with torch.no_grad():
            # Get model predictions
            logits = model(input_tensor)[0]
            next_token_logits = logits[0, -1, :] / temperature # Apply
temperature

            # Apply top-k filtering
            if top_k > 0:
                top_k_values, top_k_indices = torch.topk(next_token_logits,
top_k)
                next_token_logits = torch.full_like(next_token_logits,
float('-inf'))
                next_token_logits.scatter_(0, top_k_indices, top_k_values)

            # Apply top-p (nucleus) filtering
            if top_p < 1.0:
                sorted_logits, sorted_indices = torch.sort(next_token_logits,
descending=True)
                cumulative_probs =
torch.cumsum(torch.nn.functional.softmax(sorted_logits, dim=-1), dim=-1)

                # Remove tokens with cumulative probability above the
threshold
                sorted_indices_to_remove = cumulative_probs > top_p
                sorted_indices_to_remove[1:] = sorted_indices_to_remove[:-
1].clone()
                sorted_indices_to_remove[0] = 0

```

```

        indices_to_remove = sorted_indices[sorted_indices_to_remove]
        next_token_logits[indices_to_remove] = float('-inf')

    # Sample from the filtered distribution
    probs = torch.nn.functional.softmax(next_token_logits, dim=-1)
    next_token = torch.multinomial(probs, num_samples=1)

    # Check for end of sequence
    if next_token.item() == tokenizer.eos_token_id:
        break

    # Append to sequence
    input_tensor = torch.cat([input_tensor, next_token.unsqueeze(0)],
dim=1)

    # Prevent sequence from getting too long (memory management)
    if input_tensor.size(1) > 1024:
        input_tensor = input_tensor[:, -1024:]

    # Decode output tokens into string
    output_tokens = input_tensor[0].tolist()
    generated_text = tokenizer.decode(output_tokens,
skip_special_tokens=True)
    return generated_text

from torch import autocast, GradScaler
from transformers import GPT2TokenizerFast

torch.set_float32_matmul_precision('high')

# Initialize tokenizer first
if NOTE_TYPE == 'kaggle':
    tokenizer = GPT2TokenizerFast.from_pretrained(PATH_TO_MODEL_GPT2)
else:
    tokenizer = GPT2TokenizerFast.from_pretrained("gpt2")
tokenizer.add_special_tokens({"eos_token": "<|endoftext|>"})
tokenizer.add_special_tokens({"pad_token": "<|pad|>"})

# Use tokenizer's vocab size for the model
model = GPT(GPTConfig(vocab_size=len(tokenizer)))
model.to(device)
# if device == 'cuda':
#     torch.backends.cuda.enable_flash_sdp(True)
#     torch.backends.cuda.enable_mem_efficient_sdp(False)
#     model = torch.compile(model, mode="default") # после model.to(device)

raw_model = model

max_lr = 6e-4
min_lr = max_lr * 0.1
warmup_steps = 100
max_steps = 1000

def get_lr(it):
    if it < warmup_steps:
        return max_lr * (it+1) / warmup_steps

    if it > max_steps:

```



```

        return min_lr

    decay_ratio = (it - warmup_steps) / (max_steps - warmup_steps)
    assert 0 <= decay_ratio <= 1
    coeff = 0.5 * (1.0 + math.cos(math.pi*decay_ratio))
    return min_lr + coeff * (max_lr - min_lr)

optimizer = raw_model.configure_optimizers(weight_decay=0.1,
learning_rate=max_lr, device_type=device)

scaler = GradScaler()

# Load checkpoint if available
try:
    start_epoch = load_checkpoint(raw_model, optimizer,
path=PATH_TO_LAST_MODEL)
except:
    print("No checkpoint found, starting from scratch")
    start_epoch = 0

prompt = 'Write a Irish folktale called "The Magic Clover": '
length = 128

# Test generation with better parameters
generated_text = generate(model, tokenizer, prompt, length, temperature=0.7,
top_k=40, top_p=0.9)
print("Generated Story:")
print(generated_text)

"""**Base Model Training**

## Training Workflow

This notebook implements a two-stage training approach:

1. **Base Model Training**:
    - Uses `FolktaleDataLoader` for next-token prediction training
    - Trains the entire model from scratch (or continues from checkpoint)
    - Uses simple next-token prediction on concatenated stories
    - Higher batch size and longer training

2. **LoRA Fine-tuning**:
    - Uses `FolktalePromptDataset` for instruction-following training
    - Only trains LoRA adapters (frozen base model)
    - Uses prompt-based training with masked labels
    - Lower learning rate and shorter training

**To run the complete training pipeline:**
- First run the base model training cells
- Then apply LoRA to the trained model
- Finally run LoRA fine-tuning

**To skip base training and only do LoRA:**
- Load a pre-trained checkpoint in the model initialization cell
- Skip directly to LoRA section
"""

# Training Control Variables

```

```

# Set these flags to control which training stages to run

RUN_BASE_TRAINING = True      # Set to False if you want to skip base
                                training
RUN_LORA_TRAINING = False     # Set to False if you want to skip LoRA fine-
                                tuning
LOAD_BASE_CHECKPOINT = False  # Set to True if you want to load a base model
                                checkpoint

# Checkpoint paths
BASE_CHECKPOINT_PATH = "base_model_final.pt"
LORA_CHECKPOINT_PATH = "lora_model_final.pt"
if NOTE_TYPE == 'colab':
    DRIVE_BACKUP_DIR = r"/content/drive/MyDrive/BKP/.ipynb_checkpoints/"
else:
    DRIVE_BACKUP_DIR = r"/kaggle/working/"

print("Training Configuration:")
print(f"Run base training: {RUN_BASE_TRAINING}")
print(f"Run LoRA training: {RUN_LORA_TRAINING}")
print(f"Load base checkpoint: {LOAD_BASE_CHECKPOINT}")

if LOAD_BASE_CHECKPOINT and os.path.exists(BASE_CHECKPOINT_PATH):
    print(f"Will load base checkpoint from: {BASE_CHECKPOINT_PATH}")
elif LOAD_BASE_CHECKPOINT:
    print(f"Warning: Checkpoint {BASE_CHECKPOINT_PATH} not found!")

# Quick evaluation function
def quick_eval(model, tokenizer, device):
    """Quick evaluation with a few test prompts"""
    test_prompts = [
        "Write a Greek folktale:",
        "Write a Norse folktale about a brave warrior:",
        "Once upon a time in Ireland,"
    ]

    model.eval()
    print("\n=== Quick Evaluation ===")
    for i, prompt in enumerate(test_prompts):
        with torch.no_grad():
            result = generate(model, tokenizer, prompt, max_length=80,
                              temperature=0.8, top_k=50, top_p=0.9,
                              device=device)
            print(f"\nPrompt {i+1}: {prompt}")
            print(f"Result: {result[:200]}...")
    print("\n" * 50)

import os
import torch
import pandas as pd
import numpy as np
from tqdm import tqdm

class FolktaleDataLoader:
    def __init__(self, csv_path, tokenizer, B, T, end_token="<|endoftext|>"):
        self.B = B
        self.T = T
        self.tokenizer = tokenizer

```

```

print("Loading and tokenizing dataset...")
df = pd.read_csv(csv_path)
texts = df["text"].dropna().tolist()

# Tokenize all stories and add end-of-text token between them
token_list = []
for story in tqdm(texts, desc="Tokenizing stories"):
    tokens = tokenizer.encode(story.strip(),
add_special_tokens=False)
    tokens.append(tokenizer.eos_token_id) # Use eos_token_id
directly
    token_list.extend(tokens)

# Convert to tensor
self.tokens = torch.tensor(token_list, dtype=torch.long)
print(f"Total tokens: {len(self.tokens):,}")
print(f"1 epoch = {len(self.tokens) // (B * T)} batches")

self.current_position = 0

def next_batch(self):
    B, T = self.B, self.T
    pos = self.current_position
    next_pos = pos + B * T + 1

    if next_pos > len(self.tokens):
        # Restart from the beginning
        self.current_position = 0
        pos = self.current_position
        next_pos = pos + B * T + 1

    buf = self.tokens[pos:next_pos]
    x = buf[:-1].view(B, T)
    y = buf[1:].view(B, T)

    self.current_position += B * T
    return x, y

# Initialize base model data loader
base_train_loader = FolktaleDataLoader(
    "/kaggle/input/folk-tales-dataset/folk_tales_deduplicated.csv",
    tokenizer, B=B, T=T
)

# Base model training configuration
base_max_lr = 6e-4
base_min_lr = base_max_lr * 0.1
base_warmup_steps = 100
base_max_steps = 5000 # Training steps for base model

def get_base_lr(it):
    if it < base_warmup_steps:
        return base_max_lr * (it+1) / base_warmup_steps

    if it > base_max_steps:
        return base_min_lr

```

```

    decay_ratio = (it - base_warmup_steps) / (base_max_steps -
base_warmup_steps)
    assert 0 <= decay_ratio <= 1
    coeff = 0.5 * (1.0 + math.cos(math.pi*decay_ratio))
    return base_min_lr + coeff * (base_max_lr - base_min_lr)

# Reset model for base training (if needed)
# Uncomment the lines below if you want to train from scratch
# model = GPT(GPTConfig(vocab_size=len(tokenizer)))
# model.to(device)

# Configure optimizer for base model training
base_optimizer = model.configure_optimizers(weight_decay=0.1,
learning_rate=base_max_lr, device_type=device)

print("Base model training setup complete")
print(f"Model parameters: {sum(p.numel() for p in model.parameters()):,}")
print(f"Trainable parameters: {sum(p.numel() for p in model.parameters() if
p.requires_grad):,}")

# Base Model Training Loop
print("Starting base model training...")
print(f"Training for {base_max_steps} steps")
print(f"Gradient accumulation steps: {grad_accum_steps}")
print(f"Effective batch size: {total_batch_size}")

for step in range(base_max_steps):
    t0 = time.time()
    base_optimizer.zero_grad()
    loss_accum = 0.0
    model.train()

    for micro_step in range(grad_accum_steps):
        x, y = base_train_loader.next_batch()
        x, y = x.to(device), y.to(device)

        # Use autocast for mixed precision training
        with torch.autocast(device_type=device, dtype=torch.float16,
enabled=(device == 'cuda')):
            logits, loss = model(x, y)

        loss = loss / grad_accum_steps
        loss_accum += loss.detach()

    # Scale loss for gradient accumulation
    if device == 'cuda':
        scaler.scale(loss).backward()
    else:
        loss.backward()

    # Apply gradient clipping and optimization step
    if device == 'cuda':
        scaler.unscale_(base_optimizer)

    # Clip gradients
    norm = torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

    # Update learning rate

```

```

lr = get_base_lr(step)
for param_group in base_optimizer.param_groups:
    param_group['lr'] = lr

# Optimizer step
if device == 'cuda':
    scaler.step(base_optimizer)
    scaler.update()
else:
    base_optimizer.step()

t1 = time.time()
dt = (t1 - t0) * 1000

print(f"step {step:4d} | loss: {loss_accum.item():.6f} | lr: {lr:.2e} |
dt: {dt:.2f}ms | norm: {norm:.4f}")

# Generate sample and save checkpoint periodically
if step % 100 == 0 or step == base_max_steps - 1:
    model.eval()
    with torch.no_grad():
        # Generate a sample
        sample_prompt = "Write a Greek folktale called \"The Golden
Apple\": \"
        sample = generate(model, tokenizer, sample_prompt,
max_length=150,
                        temperature=0.8, top_k=50, top_p=0.9,
device=device)
        print(f"\n--- Sample at step {step} ---")
        print(sample[:400] + "...\" if len(sample) > 400 else sample)
        print("--- End Sample ---\n")

    # Save checkpoint every 500 steps
    if step % 100 == 0:
        save_checkpoint(model, base_optimizer, step,
f"base_model_last.pt")

print("Base model training completed!")
save_checkpoint(model, base_optimizer, base_max_steps,
f"base_model_final.pt")

"""**LoRA Fine-tuning (After Base Model Training)**"""

import torch.nn.utils.parametrize as parametrize

class LoRAParametrization(nn.Module):
    def __init__(self, features_in, features_out, rank=8, alpha=8,
device='cpu'):
        super().__init__()
        # Initialize LoRA matrices with proper initialization
        self.lora_A = nn.Parameter(torch.zeros((rank,
features_out)).to(device))
        self.lora_B = nn.Parameter(torch.zeros((features_in,
rank)).to(device))

        # Proper initialization for LoRA
        nn.init.kaiming_uniform_(self.lora_A, a=math.sqrt(5))

```

```

nn.init.zeros_(self.lora_B) # Initialize B to zero for stable
training

self.scale = alpha / rank
self.enabled = True

def forward(self, original_weights):
    if self.enabled:
        return original_weights + torch.matmul(self.lora_B,
self.lora_A).view(original_weights.shape) * self.scale
    else:
        return original_weights

def linear_layer_parametrization(layer, device, rank=16, lora_alpha=32):
    features_in, features_out = layer.weight.shape
    return LoRAParametrization(features_in, features_out, rank=rank,
alpha=lora_alpha, device=device)

for block in model.transformer.h:
    parametrize.register_parametrization(block.attn.c_attn, "weight",
linear_layer_parametrization(block.attn.c_attn, device))
    parametrize.register_parametrization(block.attn.c_proj, "weight",
linear_layer_parametrization(block.attn.c_proj, device))
    parametrize.register_parametrization(block.mlp.c_fc, "weight",
linear_layer_parametrization(block.mlp.c_fc, device))
    parametrize.register_parametrization(block.mlp.c_proj, "weight",
linear_layer_parametrization(block.mlp.c_proj, device))

def enable_disable_lora(enabled=True):
    for block in model.transformer.h:
        block.attn.c_attn.parametrizations["weight"][0].enabled = enabled
        block.attn.c_proj.parametrizations["weight"][0].enabled = enabled
        block.mlp.c_fc.parametrizations["weight"][0].enabled = enabled
        block.mlp.c_proj.parametrizations["weight"][0].enabled = enabled

count_lora = 0
count_frozen = 0
for name, param in model.named_parameters():
    if 'lora' not in name:
        param.requires_grad = False
        count_frozen += param.numel()
    else:
        count_lora += param.numel()

print(f"Forzen params {count_frozen} | Lora params {count_lora}")

enable_disable_lora(True)

# Check if LoRA is actually being applied
def debug_lora(model):
    for i, block in enumerate(model.transformer.h): # Check first 2 blocks
        c_attn = block.attn.c_attn
        print(f"Block {i} c_attn has parametrizations: {hasattr(c_attn,
'parametrizations')}")
        if hasattr(c_attn, 'parametrizations'):
            lora_layer = c_attn.parametrizations['weight'][0]
            print(f"LoRA enabled: {lora_layer.enabled}")
            print(f"LoRA A shape: {lora_layer.lora_A.shape}")

```

```

        print(f" LoRA B shape: {lora_layer.lora_B.shape}")

debug_lora(model)

# Better learning rate schedule for LoRA fine-tuning
max_lr = 3e-4 # Lower learning rate for LoRA
min_lr = max_lr * 0.1
warmup_steps = 20 # More warmup steps
max_steps = 200 # More training steps

def get_lr(it):
    if it < warmup_steps:
        return max_lr * (it+1) / warmup_steps

    if it > max_steps:
        return min_lr

    decay_ratio = (it - warmup_steps) / (max_steps - warmup_steps)
    assert 0 <= decay_ratio <= 1
    coeff = 0.5 * (1.0 + math.cos(math.pi*decay_ratio))
    return min_lr + coeff * (max_lr - min_lr)

# Only optimize LoRA parameters
lora_params = [p for n, p in model.named_parameters() if p.requires_grad and
'lora' in n]
print(f"Training {len(lora_params)} LoRA parameter tensors")

optimizer = torch.optim.AdamW(lora_params, lr=max_lr, betas=(0.9, 0.95),
eps=1e-8, weight_decay=0.01)

import os
import torch
import pandas as pd
import numpy as np
from tqdm import tqdm
import random

class FolktalePromptDataset:
    def __init__(self, csv_path, tokenizer, B, T, end_token="<|endoftext|>"):
        self.B = B
        self.T = T
        self.tokenizer = tokenizer
        self.end_token = end_token

        print("Loading and preparing dataset...")
        df = pd.read_csv(csv_path)
        df = df.dropna(subset=["text"])
        df["nation"] = df["nation"].fillna("Unknown")
        df["title"] = df["title"].fillna("Unknown Story")

        self.samples = []

        for _, row in tqdm(df.iterrows(), total=len(df), desc="Generating
prompts"):
            full_story = row["text"].strip()
            origin = row["nation"].strip()
            title = row["title"].strip()

```

```

# Clean up the story text
full_story = ' '.join(full_story.split()) # Normalize whitespace

# Task 1: Beginning a story from a prompt
prompt_1 = f"Write a {origin} folktale called \"{title}\"\\n\\n"
self.samples.append((prompt_1, full_story))

# Task 2: Continuing a story from prompt + context
sentences = full_story.split(". ")
if len(sentences) > 3:
    split_point = random.randint(1, min(len(sentences)-2, 5))
    context = ". ".join(sentences[:split_point]) + "."
    continuation = ". ".join(sentences[split_point:])
    prompt_2 = f"Continue this {origin} folktale called
    \"{title}\"\\n\\n{context}\\n\\n"
    self.samples.append((prompt_2, continuation))

print(f"Total prompt samples: {len(self.samples)}")
random.shuffle(self.samples) # Shuffle once at initialization
self.current_index = 0

def next_batch(self):
    batch_prompts = []
    batch_targets = []

    for _ in range(self.B):
        if self.current_index >= len(self.samples):
            self.current_index = 0
            random.shuffle(self.samples)

        prompt, target = self.samples[self.current_index]
        self.current_index += 1

        # Tokenize prompt and target
        prompt_ids = self.tokenizer.encode(prompt,
add_special_tokens=False)
        target_ids = self.tokenizer.encode(target,
add_special_tokens=False)
        end_token_id = self.tokenizer.encode(self.end_token)[0]

        # Combine prompt + target + end token
        full_sequence = prompt_ids + target_ids + [end_token_id]

        # Create labels: -100 for prompt tokens, actual tokens for target
        labels = [-100] * len(prompt_ids) + target_ids + [end_token_id]

        # Handle sequence length
        if len(full_sequence) > self.T:
            # Truncate but ensure we keep some target tokens
            min_prompt_len = min(len(prompt_ids), self.T // 2)
            max_target_len = self.T - min_prompt_len

            input_ids = prompt_ids[:min_prompt_len] + (target_ids +
[end_token_id])[:max_target_len]
            labels = [-100] * min_prompt_len + (target_ids +
[end_token_id])[:max_target_len]
        else:
            # Pad to sequence length

```



```

        pad_token_id = self.tokenizer.pad_token_id if
self.tokenizer.pad_token_id is not None else self.tokenizer.eos_token_id
        pad_len = self.T - len(full_sequence)
        input_ids = full_sequence + [pad_token_id] * pad_len
        labels = labels + [-100] * pad_len

        batch_prompts.append(input_ids)
        batch_targets.append(labels)

    x = torch.tensor(batch_prompts, dtype=torch.long) # input_ids
    y = torch.tensor(batch_targets, dtype=torch.long) # labels with -100
mask

    return x, y

# We'll initialize the tokenizer and dataset after model initialization

# Initialize the prompt-based dataset for LoRA fine-tuning
# This uses prompts for instruction-following fine-tuning
lora_B = 8 # Smaller batch size for LoRA
lora_T = 512

lora_train_loader = FolktalePromptDataset(
    "/kaggle/input/folk-tales-dataset/folk_tales_deduplicated.csv",
    tokenizer, B=lora_B, T=lora_T
)

# LoRA Fine-tuning Training Loop
print("Starting LoRA fine-tuning...")
print(f"Training for {max_steps} steps")
print(f"Using smaller batch size for fine-tuning: {lora_B}")

# Calculate new gradient accumulation for LoRA
lora_grad_accum_steps = total_batch_size // (lora_B * lora_T)
print(f"LoRA gradient accumulation steps: {lora_grad_accum_steps}")

for step in range(max_steps):
    t0 = time.time()
    optimizer.zero_grad()
    loss_accum = 0.0
    model.train()

    for micro_step in range(lora_grad_accum_steps):
        x, y = lora_train_loader.next_batch()
        x, y = x.to(device), y.to(device)

        # Use autocast for mixed precision training
        with torch.autocast(device_type=device, dtype=torch.float16,
enabled=(device == 'cuda')):
            logits, loss = model(x, y)

        # Only compute loss where labels are not -100
        loss = loss / lora_grad_accum_steps
        loss_accum += loss.detach()

    # Scale loss for gradient accumulation
    if device == 'cuda':
        scaler.scale(loss).backward()

```

```

        else:
            loss.backward()

    # Apply gradient clipping and optimization step
    if device == 'cuda':
        scaler.unscale_(optimizer)

    # Get the parameters that actually have gradients (LoRA parameters only)
    trainable_params = [p for p in model.parameters() if p.grad is not None
and p.requires_grad]
    norm = torch.nn.utils.clip_grad_norm_(trainable_params, 1.0)

    # Update learning rate
    lr = get_lr(step)
    for param_group in optimizer.param_groups:
        param_group['lr'] = lr

    # Optimizer step
    if device == 'cuda':
        scaler.step(optimizer)
        scaler.update()
    else:
        optimizer.step()

    t1 = time.time()
    dt = (t1 - t0) * 1000

    print(f"step {step:4d} | loss: {loss_accum.item():.6f} | lr: {lr:.2e} |
dt: {dt:.2f}ms | norm: {norm:.4f}")

    # Generate sample and save checkpoint less frequently to speed up
training
    if step % 20 == 0 or step == max_steps - 1:
        model.eval()
        with torch.no_grad():
            # Generate a sample with better parameters
            sample_prompt = "Write a Greek folktale called \"The Wise Fox\":"
            sample = generate(model, tokenizer, sample_prompt,
max_length=100,
                                temperature=0.7, top_k=40, top_p=0.9,
device=device)
            print(f"\n--- LoRA Sample at step {step} ---")
            print(sample)
            print("--- End Sample ---\n")

        # Save LoRA checkpoint
        if step % 50 == 0:
            save_checkpoint(model, optimizer, step,
f"lora_model_step_{step}.pt")

    print("LoRA fine-tuning completed!")

    save_checkpoint(model, optimizer, step, f"gpt2_lora_epoch_{step}.pt")

def evaluate_model_quality(model, tokenizer, device, num_samples=5):
    """
    Evaluate model quality with multiple prompts and different generation
parameters

```

```

"""
model.eval()

test_prompts = [
    "Write a Greek folktale called \"The Golden Apple\":",
    "Write a Norse folktale called \"The Brave Viking\":",
    "Write a Celtic folktale called \"The Magical Harp\":",
    "Write a Russian folktale called \"The Fire Bird\":",
    "Continue this Irish folktale called \"The Lucky Leprechaun\": Once
upon a time, in the rolling green hills of Ireland, there lived a tiny
leprechaun named Finn."
]

print("=== MODEL QUALITY EVALUATION ===\n")

for i, prompt in enumerate(test_prompts[:num_samples]):
    print(f"--- Test {i+1}: {prompt[:50]}... ---")

    # Test with different temperature settings
    for temp in [0.5, 0.8, 1.0]:
        print(f"\nTemperature {temp}:")
        with torch.no_grad():
            generated = generate(model, tokenizer, prompt,
max_length=150,
                                temperature=temp, top_k=50, top_p=0.9,
                                device=device)
            print(generated[:300] + "... " if len(generated) > 300 else
generated)
            print("\n" + "="*80 + "\n")

# Uncomment to run evaluation
# evaluate_model_quality(model, tokenizer, device, num_samples=3)
\end{lstlisting}

```

Приложение Б. QLoRA для qwen модели.

```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()

# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.

kvazistam_folk_tales_annotated_path =
kagglehub.dataset_download('kvazistam/folk-tales-annotated')
kvazistam_qwen_best_lora_pytorch_default_1_path =
kagglehub.model_download('kvazistam/qwen-best-lora/PyTorch/default/1')

print('Data source import complete.')
```

```
"""# Qwen2.5-7B-Instruct QLoRA Fine-tuning for Campbell Folktale
Annotation"""

!pip install -q -U transformers datasets peft trl accelerate bitsandbytes

import torch
import matplotlib.pyplot as plt

from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    BitsAndBytesConfig,
    TrainingArguments
)
import os
from transformers import TrainerCallback, EarlyStoppingCallback
from datasets import load_dataset
from peft import LoraConfig, prepare_model_for_kbit_training, get_peft_model,
PeftModel
from trl import SFTTrainer
from huggingface_hub import login

PATH_TO_CSV = "/kaggle/input/datasets/kvazistam/folk-tales-
annotated/folk_tales_annotated.csv"
LORA_PATH = "/kaggle/input/models/kvazistam/qwen-best-
lora/pytorch/default/1/best_lora"
HF_TOKEN = "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"
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Device: {DEVICE}")
login(HF_TOKEN)

# DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

# MODEL_NAME = "Qwen/Qwen2.5-7B-Instruct"

# bnb_config = BitsAndBytesConfig(
#     load_in_4bit=True,
```

```

#     bnb_4bit_quant_type="nf4",
#     bnb_4bit_compute_dtype=torch.bfloat16,
#     bnb_4bit_use_double_quant=True
# )

tokenizer = AutoTokenizer.from_pretrained(
#     MODEL_NAME,
#     trust_remote_code=True
# )

model = AutoModelForCausalLM.from_pretrained(
#     MODEL_NAME,
#     quantization_config=bnb_config,
#     device_map="auto",
#     trust_remote_code=True
# )
model.gradient_checkpointing_enable()

import os
import torch

from transformers import AutoModelForCausalLM, AutoTokenizer,
BitsAndBytesConfig
from peft import (
    PeftModel,
    LoraConfig,
    get_peft_model,
    prepare_model_for_kbit_training
)

def load_qwen_with_lora(
    model_name="Qwen/Qwen2.5-7B-Instruct",
    lora_path="./best_lora",
    r=64,
    lora_alpha=128,
    lora_dropout=0.05,
    target_modules=None
):
    if target_modules is None:
        target_modules = [
            "q_proj", "k_proj", "v_proj", "o_proj",
            "gate_proj", "up_proj", "down_proj"
        ]

    bnb_config = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_quant_type="nf4",
        bnb_4bit_compute_dtype=torch.bfloat16,
        bnb_4bit_use_double_quant=True
    )

    tokenizer = AutoTokenizer.from_pretrained(
        model_name,
        trust_remote_code=True

```

```

)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    device_map="auto",
    quantization_config=bnb_config,
    trust_remote_code=True
)

if os.path.exists(lora_path):

    print(f"🔄 RESUME LoRA from: {lora_path}")

    model = PeftModel.from_pretrained(
        model,
        lora_path,
        is_trainable=True
    )

else:

    print(f"🆕 Creating new LoRA adapter")

    model = prepare_model_for_kbit_training(model)

    lora_config = LoraConfig(
        r=r,
        lora_alpha=lora_alpha,
        lora_dropout=lora_dropout,
        bias="none",
        task_type="CAUSAL_LM",
        target_modules=target_modules
    )

    model = get_peft_model(model, lora_config)

    model.train()
    model.gradient_checkpointing_enable()
    model.config.use_cache = False
    return model, tokenizer

model, tokenizer = load_qwen_with_lora(
    model_name="Qwen/Qwen2.5-7B-Instruct",
    lora_path=LORA_PATH
)

model.print_trainable_parameters()

# model.train()

# model = prepare_model_for_kbit_training(model)
# #
# model.gradient_checkpointing_enable()
# model.config.use_cache = False

```

```
# model.print_trainable_parameters()

import pandas as pd
import re
from datasets import load_dataset, Dataset
df = pd.read_csv(PATH_TO_CSV)
df = df[["title", "nation", "text"]]
df["annotated_text"] = df["text"]
df["text"] = df["text"].apply(lambda x: re.sub(r'<.*?>', '', x))
df.head()
dataset = Dataset.from_pandas(df)

dataset

def build_chat(example):

    messages = [
        {
            "role": "system",
            "content": (
                "You are an expert in Joseph Campbell's Hero Journey. "
                "Do not modify text outside inserted stage tags."
            )
        },
        {
            "role": "user",
            "content": f'''Annotate the folktale using Campbell stages.

stages (use THIS tegs)
### Departure
<stage_call_to_adventure> | <stage_refusal_of_call> |
<stage_supernatural_aid> | <stage_crossing_threshold> |
<stage_belly_of_whale>
### Initiation
<stage_road_of_trials> | <stage_meeting_goddess> | <stage_woman_as_temptress>
| <stage_atonement_father> | <stage_apotheosis> | <stage_ultimate_boon>
### Return
<stage_refusal_return> | <stage_magic_flight> | <stage_rescue_from_without> |
<stage_crossing_return_threshold> | <stage_master_two_worlds> |
<stage_freedom_to_live>

Title: {example["title"]}
Nation: {example["nation"]}

Folktale:

{example["text"]}'''
        ]

    text = tokenizer.apply_chat_template(
        messages,
        tokenize=False
    )
```

```

    return {"text": text}

dataset = dataset.map(
    build_chat,
    remove_columns=dataset.column_names
)

dataset["text"][0][:50]

new_dataset = dataset.train_test_split(
    test_size=0.1,
    seed=42
)

train_dataset = new_dataset["train"]
eval_dataset = new_dataset["test"]
train_dataset, eval_dataset

# from peft import (
#     LoraConfig,
#     prepare_model_for_kbit_training,
#     get_peft_model
# )

# lora_config = LoraConfig(
#     r=64,
#     lora_alpha=128,
#     lora_dropout=0.05,
#     bias="none",
#     task_type="CAUSAL_LM",
#     target_modules=[
#         "q_proj",
#         "k_proj",
#         "v_proj",
#         "o_proj",
#         "gate_proj",
#         "up_proj",
#         "down_proj"
#     ]
# )

# model = get_peft_model(
#     model,
#     lora_config
# )

# model.print_trainable_parameters()

"""## НОВОЕ"""

from transformers import TrainerCallback
from IPython.display import clear_output
import matplotlib.pyplot as plt

class LiveLossPlotCallback(TrainerCallback):

```



```

def __init__(self):
    self.steps = []
    self.losses = []

def on_log(self, args, state, control, logs=None, **kwargs):

    if logs is None:
        return

    if "loss" not in logs:
        return

    self.steps.append(state.global_step)
    self.losses.append(logs["loss"])

    clear_output(wait=True)

    plt.figure(figsize=(8, 4))
    plt.plot(self.steps, self.losses)
    plt.title("Training Loss")
    plt.xlabel("Global Step")
    plt.ylabel("Loss")
    plt.grid(True)
    plt.show()

from transformers import EarlyStoppingCallback

early_stopping = EarlyStoppingCallback(
    early_stopping_patience=2,
    early_stopping_threshold=0.001
)

from datetime import datetime
class HardDebugCallback(TrainerCallback):

    def on_step_end(self, args, state, control, **kwargs):
        if state.global_step % 1 == 0:
            print(f"STEP {state.global_step} | time:
{datetime.now().strftime('%H-%M-%S')}")

"""## Обучение"""

from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="./qwen_campbell",

    num_train_epochs=5,

    per_device_train_batch_size=1,
    gradient_accumulation_steps=16,

    learning_rate=2e-4,
    warmup_ratio=0.05,
    lr_scheduler_type="cosine",

    logging_steps=1,
    logging_first_step=True,

```

```

eval_strategy="epoch",
save_strategy="epoch",
logging_strategy="steps",

load_best_model_at_end=True,

metric_for_best_model="eval_loss",
greater_is_better=False,

save_total_limit=2,

bf16=True,

optim="paged_adamw_8bit",

report_to="none",

disable_tqdm=False
)

from transformers import TrainingArguments
from trl import SFTTrainer

live_callback = LiveLossPlotCallback()
hard_debug = HardDebugCallback()
trainer = SFTTrainer(
    model=model,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    processing_class=tokenizer,
    args=training_args,
    callbacks=[
        live_callback,
        early_stopping,
        hard_debug
    ]
)

trainer.train()

trainer.model.save_pretrained(
    "/kaggle/working/qwen_campbell_lora"
)

tokenizer.save_pretrained(
    "/kaggle/working/qwen_campbell_lora"
)

print("Best checkpoint:")
print(trainer.state.best_model_checkpoint)

print("Best eval loss:")
print(trainer.state.best_metric)

prompt = '''
Generate story about girl, who planet traveler
'''

```

```

messages = [
    {"role": "user", "content": prompt}
]

text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True
)

inputs = tokenizer(
    text,
    return_tensors="pt"
).to(model.device)

outputs = model.generate(
    **inputs,
    max_new_tokens=1024,
    temperature=0.1
)

print(tokenizer.decode(outputs[0], skip_special_tokens=True))

```

Приложение В. Модифицированный LoRA для SLM.

```
# -*- coding: utf-8 -*-

# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()

# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.

kvazistam_lora_dataset_path = kagglehub.dataset_download('kvazistam/lora-
dataset')

print('Data source import complete.')

"""## Preprocess"""

#GLOBALS
PATH_TOKENIZER = "/kaggle/input/slm-files/folktale_tokenizer.json"
PATH_MODEL = "/kaggle/input/slm-files/slm_model_step_5000.pt"
PATH_DATASET = "/kaggle/input/datasets/kvazistam/lora-
dataset/folk_tales_annotated.csv"
PATH_LORA_MODEL = "/kaggle/working/lora-slm-storyteller-new.pth"

import pandas as pd
from tokenizers import Tokenizer
from tokenizers.models import BPE
from tokenizers.trainers import BpeTrainer
from tokenizers.pre_tokenizers import ByteLevel
from tokenizers.processors import TemplateProcessing
from tokenizers.decoders import ByteLevel as ByteLevelDecoder
from transformers import PreTrainedTokenizerFast
from pathlib import Path

def load_custom_tokenizer(tokenizer_path="custom_tokenizer.json"):
    """
    Load the custom trained tokenizer and wrap it with
    PreTrainedTokenizerFast
    """
    tokenizer = Tokenizer.from_file(tokenizer_path)

    # Wrap with PreTrainedTokenizerFast for compatibility
    fast_tokenizer = PreTrainedTokenizerFast(
        tokenizer_object=tokenizer,
        unk_token="<|unk|>",
        pad_token="<|pad|>",
        eos_token="<|endoftext|>",
```

```

        bos_token="<|startoftext|>",
    )

    return fast_tokenizer

from dataclasses import dataclass
import torch
import torch.nn as nn
from torch.nn import functional as F
import math
import inspect
import tiktoken
import os
from torch.distributed import init_process_group, destroy_process_group
from torch.nn.parallel import DistributedDataParallel as DDP
import torch.distributed as dist
import numpy as np
import pickle
import random
import torch.multiprocessing as mp
import time

class CausalSelfAttention(nn.Module):

    def __init__(self, config):
        super().__init__()
        assert config.n_embed % config.n_head == 0
        self.c_attn = nn.Linear(config.n_embed, 3 * config.n_embed)
        self.c_proj = nn.Linear(config.n_embed, config.n_embed)
        self.c_proj.NANO_GPT_SCALE_INIT = 1
        self.n_head = config.n_head
        self.n_embed = config.n_embed
        # Add dropout for regularization
        self.dropout = nn.Dropout(config.dropout)

    def forward(self, x, attention_mask = None):
        B, T, C = x.size()
        qkv = self.c_attn(x)
        q, k, v = qkv.split(self.n_embed, dim=2)
        k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
        q = q.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)
        v = v.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)

        if attention_mask is not None:
            # expand mask to [B, 1, 1, T] so it can broadcast to [B,
            num_heads, T, T]
            attn_mask = attention_mask[:, None, None, :].to(q.dtype)
        else:
            attn_mask = None

        y = F.scaled_dot_product_attention(q, k, v, is_causal=True,
            attn_mask=attn_mask, dropout_p=0.1 if self.training else 0.0)

        y = y.transpose(1, 2).contiguous().view(B, T, C)
        y = self.c_proj(y)
        return y

```

```

class MLP(nn.Module):

    def __init__(self, config):
        super().__init__()
        self.c_fc = nn.Linear(config.n_embed, 4 * config.n_embed)
        self.gelu = nn.GELU(approximate='tanh')
        self.c_proj = nn.Linear(4 * config.n_embed, config.n_embed)
        self.c_proj.NANOGPT_SCALE_INIT = 1
        self.dropout = nn.Dropout(config.dropout)

    def forward(self, x):
        x = self.c_fc(x)
        x = self.gelu(x)
        x = self.dropout(self.c_proj(x))
        return x

```

```

class Block(nn.Module):

    def __init__(self, config):
        super().__init__()
        self.ln_1 = nn.LayerNorm(config.n_embed)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = nn.LayerNorm(config.n_embed)
        self.mlp = MLP(config)

    def forward(self, x, mask = None):
        x = x + self.attn(self.ln_1(x), mask)
        x = x + self.mlp(self.ln_2(x))
        return x

```

```

@dataclass
class GPTConfig:
    block_size: int = 512
    vocab_size: int = 20000
    n_layer: int = 8
    n_head: int = 8
    n_embed: int = 768
    dropout: float = 0.1 # Add dropout configuration

```

```

class GPT(nn.Module):

    def __init__(self, config):
        super().__init__()
        self.config = config
        self.transformer = nn.ModuleDict(dict(
            wte = nn.Embedding(config.vocab_size, config.n_embed),
            wpe = nn.Embedding(config.block_size, config.n_embed),
            drop = nn.Dropout(config.dropout), # Add input dropout
            h = nn.ModuleList([Block(config) for _ in
range(config.n_layer)]),
            ln_f= nn.LayerNorm(config.n_embed)
        ))
        self.lm_head = nn.Linear(config.n_embed, config.vocab_size,
bias=False)

        # weight sharing

```

```

self.transformer.wte.weight = self.lm_head.weight

# init weights
self.apply(self._init_weights)

def _init_weights(self, module):
    if isinstance(module, nn.Linear):
        std = 0.02
        if hasattr(module, 'NANO_GPT_SCALE_INIT'):
            std *= (2 * self.config.n_layer) ** -0.5
        torch.nn.init.normal_(module.weight, mean=0.0, std=std)
        if module.bias is not None:
            torch.nn.init.zeros_(module.bias)
    elif isinstance(module, nn.Embedding):
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)

def forward(self, idx, targets=None, attention_mask=None):
    B, T = idx.size()
    assert T <= self.config.block_size

    pos = torch.arange(0, T, dtype=torch.long, device=idx.device)
    pos_emb = self.transformer.wpe(pos)
    tok_emb = self.transformer.wte(idx)
    x = self.transformer.drop(tok_emb + pos_emb) # Apply input dropout
    for block in self.transformer.h:
        x = block(x, attention_mask)
    x = self.transformer.ln_f(x)
    logits = self.lm_head(x)
    loss = None
    if targets is not None:
        loss = F.cross_entropy(
            logits.view(-1, logits.size(-1)),
            targets.view(-1),
            ignore_index=-100
        )
    return logits, loss

def configure_optimizers(self, weight_decay, learning_rate, device_type):
    # start with all of the candidate parameters (that require grad)
    param_dict = {pn: p for pn, p in self.named_parameters()}
    param_dict = {pn: p for pn, p in param_dict.items() if
p.requires_grad}
    # create optim groups. Any parameters that is 2D will be weight
    decayed, otherwise no.
    # i.e. all weight tensors in matmuls + embeddings decay, all biases
    and layernorms don't.
    decay_params = [p for n, p in param_dict.items() if p.dim() >= 2]
    nodecay_params = [p for n, p in param_dict.items() if p.dim() < 2]
    optim_groups = [
        {'params': decay_params, 'weight_decay': weight_decay},
        {'params': nodecay_params, 'weight_decay': 0.0}
    ]
    num_decay_params = sum(p.numel() for p in decay_params)
    num_nodecay_params = sum(p.numel() for p in nodecay_params)
    print(f"num decayed parameter tensors: {len(decay_params)}, with
{num_decay_params:,} parameters")
    print(f"num non-decayed parameter tensors: {len(nodecay_params)},
with {num_nodecay_params:,} parameters")

```

```

        # Create AdamW optimizer and use the fused version if it is available
        fused_available = 'fused' in
inspect.signature(torch.optim.AdamW).parameters
        use_fused = fused_available and device_type == "cuda"
        optimizer = torch.optim.AdamW(optim_groups, lr=learning_rate,
betas=(0.9, 0.95), eps=1e-8, fused=use_fused)
        return optimizer

def load_checkpoint(model, optimizer, path="model_checkpoint.pt"):
    checkpoint = torch.load(path, weights_only=True)

    model.load_state_dict(checkpoint['model_state_dict'])
    optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
    epoch = checkpoint['epoch']

    print(f"Checkpoint loaded: Resuming from epoch {epoch}")
    return epoch

def save_checkpoint(model, optimizer, epoch, path="model_checkpoint.pt"):
    torch.save({
        'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
    }, path)
    print(f"Model checkpoint saved at epoch {epoch}")

def generate_sample(model, tokenizer, max_length=64, device='cuda' if
torch.cuda.is_available() else 'cpu'):
    # Step 1: Initialize prompt
    prompt = "Write a greek folktale called \"The Dog and the Wolf\" \n"
    return generate(model, tokenizer, prompt, max_length, temperature=0.8,
top_k=50, top_p=0.9, device=device)

"""
=====
=====
"""

device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f"Using device: {device}")

torch.manual_seed(1337)
if torch.cuda.is_available():
    torch.cuda.manual_seed(1337)

total_batch_size = 64 * 512
B = 64
T = 512
assert total_batch_size % (B * T) == 0
grad_accum_steps = total_batch_size // (B * T)

print(f"total desired batch size: {total_batch_size}")
print(f"> calculated gradient accumulation steps: {grad_accum_steps}")

```



```

def generate(model, tokenizer, prompt, max_length=50, temperature=0.8,
top_k=50, top_p=0.9, device='cuda' if torch.cuda.is_available() else 'cpu'):
    """
    Improved generation function with better sampling strategies
    """
    input_ids = tokenizer.encode(prompt.strip(), add_special_tokens=False)
    input_tensor = torch.tensor([input_ids], dtype=torch.long).to(device)

    # Switch to eval mode
    model.eval()

    # Autoregressive token generation
    for _ in range(max_length):
        with torch.no_grad():
            # Get model predictions
            logits = model(input_tensor)[0]
            next_token_logits = logits[0, -1, :] / temperature # Apply
temperature

            # Apply top-k filtering
            if top_k > 0:
                top_k_values, top_k_indices = torch.topk(next_token_logits,
top_k)
                next_token_logits = torch.full_like(next_token_logits,
float('-inf'))
                next_token_logits.scatter_(0, top_k_indices, top_k_values)

            # Apply top-p (nucleus) filtering
            if top_p < 1.0:
                sorted_logits, sorted_indices = torch.sort(next_token_logits,
descending=True)
                cumulative_probs =
torch.cumsum(torch.nn.functional.softmax(sorted_logits, dim=-1), dim=-1)

                # Remove tokens with cumulative probability above the
threshold
                sorted_indices_to_remove = cumulative_probs > top_p
                sorted_indices_to_remove[1:] = sorted_indices_to_remove[:-
1].clone()
                sorted_indices_to_remove[0] = 0

                indices_to_remove = sorted_indices[sorted_indices_to_remove]
                next_token_logits[indices_to_remove] = float('-inf')

            # Sample from the filtered distribution
            probs = torch.nn.functional.softmax(next_token_logits, dim=-1)
            next_token = torch.multinomial(probs, num_samples=1)

            # Check for end of sequence
            if next_token.item() == tokenizer.eos_token_id:
                break

            # Append to sequence
            input_tensor = torch.cat([input_tensor, next_token.unsqueeze(0)],
dim=1)

            # Prevent sequence from getting too long (memory management)
            if input_tensor.size(1) > 1024:

```

```

        input_tensor = input_tensor[:, -1024:]

    # Decode output tokens into string
    output_tokens = input_tensor[0].tolist()
    generated_text = tokenizer.decode(output_tokens,
    skip_special_tokens=False)
    return generated_text

from torch import autocast, GradScaler
from transformers import GPT2TokenizerFast

torch.set_float32_matmul_precision('high')

# Initialize tokenizer first
tokenizer = load_custom_tokenizer(PATH_TOKENIZER)

# Use tokenizer's vocab size for the model
model = GPT(GPTConfig(vocab_size=len(tokenizer)))
model.to(device)
raw_model = model

max_lr = 1e-3
max_steps = 100

optimizer = raw_model.configure_optimizers(weight_decay=0.1,
learning_rate=max_lr, device_type=device)

scaler = GradScaler()

start_epoch = load_checkpoint(raw_model, optimizer, path=PATH_MODEL)

"""## **LoRA Fine-tuning (After Base Model Training)**"""

import torch.nn.utils.parametrize as parametrize

class LoRAParametrization(nn.Module):
    def __init__(self, features_in, features_out, rank=8, alpha=8,
device='cpu'):
        super().__init__()
        # Initialize LoRA matrices with proper initialization
        self.lora_A = nn.Parameter(torch.zeros((rank,
features_out)).to(device))
        self.lora_B = nn.Parameter(torch.zeros((features_in,
rank)).to(device))

        # Proper initialization for LoRA
        nn.init.kaiming_uniform_(self.lora_A, a=math.sqrt(5))
        nn.init.zeros_(self.lora_B) # Initialize B to zero for stable
training

        self.scale = alpha / rank
        self.enabled = True

    def forward(self, original_weights):
        if self.enabled:
            return original_weights + torch.matmul(self.lora_B,
self.lora_A).view(original_weights.shape) * self.scale
        else:

```

```

        return original_weights

def linear_layer_parametrization(layer, device, rank=4, lora_alpha=8):
    features_in, features_out = layer.weight.shape
    return LoRAParametrization(features_in, features_out, rank=rank,
                                alpha=lora_alpha, device=device)

for block in model.transformer.h:
    parametrize.register_parametrization(block.attn.c_attn, "weight",
    linear_layer_parametrization(block.attn.c_attn, device))
    parametrize.register_parametrization(block.attn.c_proj, "weight",
    linear_layer_parametrization(block.attn.c_proj, device))
    parametrize.register_parametrization(block.mlp.c_fc, "weight",
    linear_layer_parametrization(block.mlp.c_fc, device))
    parametrize.register_parametrization(block.mlp.c_proj, "weight",
    linear_layer_parametrization(block.mlp.c_proj, device))

def enable_lora(enabled=True):
    for block in model.transformer.h:
        block.attn.c_attn.parametrizations["weight"][0].enabled = enabled
        block.attn.c_proj.parametrizations["weight"][0].enabled = enabled
        block.mlp.c_fc.parametrizations["weight"][0].enabled = enabled
        block.mlp.c_proj.parametrizations["weight"][0].enabled = enabled

count_lora = 0
count_frozen = 0

for name, param in model.named_parameters():
    if 'lora' not in name:
        param.requires_grad = False
        count_frozen += param.numel()
    else:
        count_lora += param.numel()

print(f"Forzen params {count_frozen} | Lora params {count_lora}")

enable_lora(True)

# Check if LoRA is actually being applied
def debug_lora(model):
    for i, block in enumerate(model.transformer.h): # Check first 2 blocks
        c_attn = block.attn.c_attn
        print(f"Block {i} c_attn has parametrizations: {hasattr(c_attn,
        'parametrizations')}")
        if hasattr(c_attn, 'parametrizations'):
            lora_layer = c_attn.parametrizations['weight'][0]
            print(f"    LoRA enabled: {lora_layer.enabled}")
            print(f"    LoRA A shape: {lora_layer.lora_A.shape}")
            print(f"    LoRA B shape: {lora_layer.lora_B.shape}")

debug_lora(model)

# Better learning rate schedule for LoRA fine-tuning
max_lr = 5e-5 # Lower learning rate for LoRA
max_steps = 200 # More training steps

# Only optimize LoRA parameters

```

```

lora_params = [p for n, p in model.named_parameters() if p.requires_grad and
'loa' in n]
print(f"Training {len(lora_params)} LoRA parameter tensors")

optimizer = torch.optim.AdamW(lora_params, lr=max_lr, betas=(0.9, 0.95),
eps=1e-8, weight_decay=0.01)

import os
import torch
import pandas as pd
import random
from tqdm import tqdm

class FolktalesPromptDataset:
    def __init__(self, csv_path, tokenizer, B, prompt_len=16, answer_len=496,
end_token="<|endoftext|>"):
        self.B = B
        self.prompt_len = prompt_len
        self.answer_len = answer_len
        self.T = prompt_len + answer_len
        self.tokenizer = tokenizer
        self.end_token = end_token

        print("Loading and preparing dataset...")
        df = pd.read_csv(csv_path)
        df = df.dropna(subset=["text"])
        df["nation"] = df["nation"].fillna("Unknown")
        df["title"] = df["title"].fillna("Unknown Story")

        self.samples = []

        for _, row in tqdm(df.iterrows(), total=len(df), desc="Generating
prompts"):
            full_story = ' '.join(str(row["text"]).split())
            title = row["title"].strip()

            # Construct prompt and target
            prompt = f"TITLE: {title}. \n\n" # keep title + double newline
            target = full_story.strip()

            self.samples.append((prompt, target))

        print(f"Total prompt samples: {len(self.samples)}")
        random.shuffle(self.samples)
        self.current_index = 0

    def next_batch(self):
        batch_inputs, batch_labels, batch_attention_mask = [], [], []

        for _ in range(self.B):
            if self.current_index >= len(self.samples):
                self.current_index = 0
                random.shuffle(self.samples)

            prompt, target = self.samples[self.current_index]
            self.current_index += 1

            # --- Encode ---

```

```

        prompt_ids = self.tokenizer.encode(prompt,
add_special_tokens=False)
        target_ids = self.tokenizer.encode(target,
add_special_tokens=False)
        end_token_id = self.tokenizer.encode(self.end_token,
add_special_tokens=False)[0]

        # --- Truncate / pad prompt to fixed length ---
        if len(prompt_ids) > self.prompt_len:
            prompt_ids = prompt_ids[:self.prompt_len]
        else:
            pad_len = self.prompt_len - len(prompt_ids)
            pad_token_id = self.tokenizer.pad_token_id or
self.tokenizer.eos_token_id
            prompt_ids = prompt_ids + [pad_token_id] * pad_len

        # --- Truncate / pad target to fixed length ---
        if len(target_ids) > self.answer_len - 1: # reserve 1 token for
end_token
            target_ids = target_ids[:self.answer_len - 1]
            target_ids = target_ids + [end_token_id] # append end token
        if len(target_ids) < self.answer_len:
            pad_len = self.answer_len - len(target_ids)
            pad_token_id = self.tokenizer.pad_token_id or
self.tokenizer.eos_token_id
            target_ids = target_ids + [pad_token_id] * pad_len

        # --- Combine prompt + target ---
        input_ids = prompt_ids + target_ids

        # --- Labels: -100 for prompt and padding tokens ---
        labels = [-100]*self.prompt_len
        for t in target_ids:
            if t == (self.tokenizer.pad_token_id or
self.tokenizer.eos_token_id):
                labels.append(-100)
            else:
                labels.append(t)

        # --- Attention mask: 1 for non-padding, 0 for padding ---
        attention_mask = [1 if t != (self.tokenizer.pad_token_id or
self.tokenizer.eos_token_id) else 0
                        for t in input_ids]

        batch_inputs.append(input_ids)
        batch_labels.append(labels)
        batch_attention_mask.append(attention_mask)

    x = torch.tensor(batch_inputs, dtype=torch.long)
    y = torch.tensor(batch_labels, dtype=torch.long)
    mask = torch.tensor(batch_attention_mask, dtype=torch.long)
    return x, y, mask

# Initialize the prompt-based dataset for LoRA fine-tuning
# This uses prompts for instruction-following fine-tuning
lora_B = 32 # Smaller batch size for LoRA
lora_T = 512

```

```

lora_train_loader = FolktalePromptDataset(
    PATH_DATASET,
    tokenizer, B=lora_B
)

def repetition_penalty_loss(logits, input_ids, window=25):
    """
    Penalize if model assigns high probability to tokens it already produced
    recently.
    Vectorized version (much faster).
    """
    B, T, V = logits.shape
    probs = F.softmax(logits, dim=-1)

    # For each position, gather probabilities assigned to each input token
    # Shape: [B, T, window]
    gather_indices = torch.zeros((B, T, window), dtype=torch.long,
device=logits.device)
    for w in range(window):
        # Shifted view of input ids, pad with zeros
        pad = torch.zeros((B, w), dtype=torch.long, device=input_ids.device)
        rolled = torch.cat([pad, input_ids[:, :-w] if w > 0 else input_ids],
dim=1)
        gather_indices[:, :, w] = rolled

    # Gather probs of those recently used tokens
    gathered = torch.gather(probs, 2, gather_indices)
    # Mean probability assigned to recent tokens
    penalty = gathered.mean()

    return penalty

# LoRA Fine-tuning Training Loop
print("Starting LoRA fine-tuning...")
print(f"Training for {max_steps} steps")
print(f"Using smaller batch size for fine-tuning: {lora_B}")

# Calculate new gradient accumulation for LoRA
lora_grad_accum_steps = total_batch_size // (lora_B * lora_T)
print(f"LoRA gradient accumulation steps: {lora_grad_accum_steps}")

for step in range(max_steps):
    t0 = time.time()
    optimizer.zero_grad()
    loss_accum = 0.0
    model.train()

    for micro_step in range(lora_grad_accum_steps):
        x, y, mask = lora_train_loader.next_batch()
        x, y, mask = x.to(device), y.to(device), mask.to(device)

        # Use autocast for mixed precision training
        with torch.autocast(device_type=device, dtype=torch.float16):
            logits, loss = model(x, y, mask)

            loss += 2 * repetition_penalty_loss(logits, x)

    # Only compute loss where labels are not -100

```

```

        loss = loss / lora_grad_accum_steps
        loss_accum += loss.detach()

    scaler.scale(loss).backward()

    # Apply gradient clipping and optimization step
    scaler.unscale_(optimizer)

    # Get the parameters that actually have gradients (LoRA parameters only)
    trainable_params = [p for p in model.parameters() if p.grad is not None
and p.requires_grad]
    norm = torch.nn.utils.clip_grad_norm_(trainable_params, 1.0)

    scaler.step(optimizer)
    scaler.update()

    t1 = time.time()
    dt = (t1 - t0) * 1000

    print(f"step {step:4d} | loss: {loss_accum.item():.6f} | dt: {dt:.2f}ms |
norm: {norm:.4f}")

    # Generate sample and save checkpoint less frequently to speed up
    training
    if step % 50 == 0 or step == max_steps - 1:
        model.eval()
        with torch.no_grad():
            # Generate a sample with better parameters
            sample_prompt = f"TITLE: Down the rabbit hole. \n\n"
            sample = generate(model, tokenizer, sample_prompt,
max_length=100,
                                temperature=0.7, top_k=40, top_p=0.9,
device=device)
            print(f"\n--- LoRA Sample at step {step} ---")
            print(sample)
            print("--- End Sample ---\n")

        # Save LoRA checkpoint
        if step % 50 == 0:
            save_checkpoint(model, optimizer, step,
f"lora_model_step_{step}.pt")

    print("LoRA fine-tuning completed!")

length = 256

prompt = "Tell me a story about a little girl on another planet. Insert
monomith stages in text\n call to adventure "
# prompt = "stage_call_to_adventure There were, once upon a time, a king and
queen of Denmark who had an only son,"
output = generate(model, tokenizer, prompt, max_length=length,
temperature=1.0, top_k=0, top_p=0.90)
print(output)

"""## Поэтапная генерация"""

import torch

```

```

def generate_campbell_story(base_prompt, model, tokenizer, device='cuda',
tokens_per_stage=50):
    """
    Генерирует сказку поэтапно, используя выученные теги <stage_...>
    """
    # Последовательность этапов, через которую мы проведем сюжет.
    # Можно сократить или изменить порядок под свой вкус.
    stages = [
        "call to adventure",      # Зов к приключениям
        "refusal of call",        # Отказ от зова (или страх)
        "supernatural aid",       # Сверхъестественная помощь
        "crossing threshold",     # Переход порога
        "road of trials",         # Дорога испытаний
        "meeting goddess",       # Встреча с богиней/помощником
        "ultimate boon",         # Добыча эликсира (цель)
        "master two worlds",     # Владыка двух миров
        "freedom to live"        # Свобода жить
    ]

    current_text = base_prompt + "\n\n"

    print(f"Начинаем историю: '{base_prompt}'")
    print("=" * 60)

    for i, stage_name in enumerate(stages):
        stage_token = f"stage {stage_name}"

        # Добавляем токен этапа в контекст
        current_text += f"{stage_token} "

        # --- Генерация куска текста для этого этапа ---
        print(f"\n ♦ Этап: {stage_name.upper().replace('_', ' ')}")

        # Мы генерируем, передавая весь накопленный текст как контекст
        # generate вернет: [Prompt + Generated Text]
        output_sequence = generate(model, tokenizer, current_text,
                                max_length=tokens_per_stage,
                                temperature=0.7,
                                top_k=40,
                                device=device)

        # Вырезаем только новую часть (убираем промпт)
        # generate обычно возвращает строку целиком
        new_text = output_sequence[len(current_text):]

        # Чистка: убираем случайные повторения токенов, если модель
        "заикнулась"
        if stage_token in new_text:
            new_text = new_text.split(stage_token)[0]

        # Если модель сама сгенерировала СЛЕДУЮЩИЙ токен, обрезаем там
        next_stage_token = f"<stage_{stages[i+1]}>" if i < len(stages) - 1
    else None
        if next_stage_token and next_stage_token in new_text:
            new_text = new_text.split(next_stage_token)[0]

    new_text = new_text.strip()
    print(f"{new_text}")

```



```

        # Обновляем текущий текст для следующего шага
        current_text += new_text + " "

    print("\n" + "=" * 60)
    print("История завершена.")
    return current_text

# Пример вызова
prompt = "Write a fairy tale about a girl astronaut discovering a secret on Mars."

# Запускаем генерацию
full_story = generate_campbell_story(prompt, model, tokenizer, device='cuda',
tokens_per_stage=150)

import torch

def save_lora_parameters(model, path):
    lora_state = {}
    for name, module in model.named_modules():
        if hasattr(module, "parametrizations") and "weight" in
module.parametrizations:
            lora = module.parametrizations["weight"][0]
            if isinstance(lora, LoRAParametrization):
                lora_state[f"{name}.lora_A"] = lora.lora_A.detach().cpu()
                lora_state[f"{name}.lora_B"] = lora.lora_B.detach().cpu()
                lora_state[f"{name}.scale"] = torch.tensor(lora.scale)
    torch.save(lora_state, path)
    print(f"Saved LoRA parameters to {path}")

def load_lora_parameters(model, path, device="cpu"):
    lora_state = torch.load(path, map_location=device, weights_only=True)
    for name, module in model.named_modules():
        if hasattr(module, "parametrizations") and "weight" in
module.parametrizations:
            lora = module.parametrizations["weight"][0]
            if isinstance(lora, LoRAParametrization):
                lora.lora_A.data = lora_state[f"{name}.lora_A"].to(device)
                lora.lora_B.data = lora_state[f"{name}.lora_B"].to(device)
                lora.scale = lora_state[f"{name}.scale"].item()
    print(f"Loaded LoRA parameters from {path}")

save_lora_parameters(model, PATH_LORA_MODEL)

load_lora_parameters(model, PATH_LORA_MODEL, "cuda")

```